

第14章 智能目标识别与跟踪



全红艳

计算机科学与技术学院

本次课程内容

1. 目标识别知识回顾

2. 目标识别问题

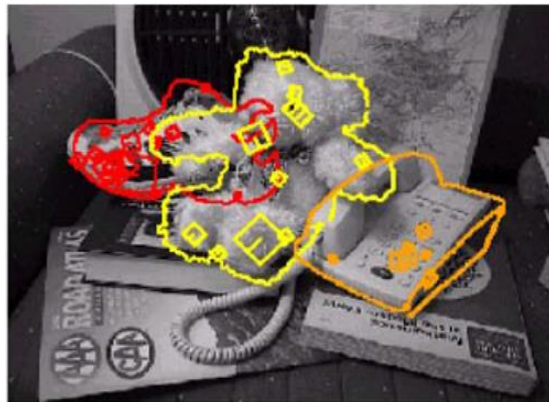
3. R-CNN系列算法

4. YOLO系列算法

1. 相关知识的回顾

(1) 目标识别知识回顾

- ◆ **目标识别**:是指从一幅图像中, 识别出一个特殊目标或检查出一类目标。



目标级的识别

- ◆ 从识别范围分类
 - 图像级目标识别
 - 目标级的识别
 - 像素级的目标识别（像素级别识别）
 - 实例目标分割

猫的图片



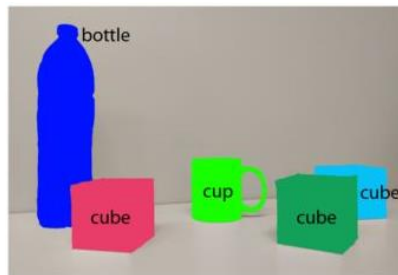
图像级目标识别



像素级目标识别



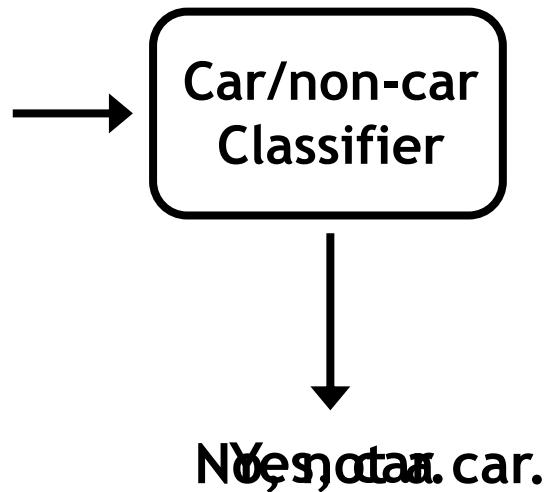
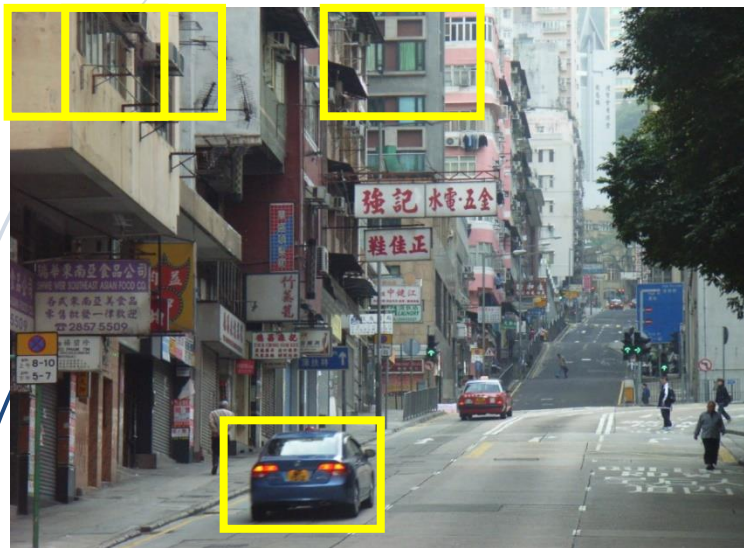
目标级的识别



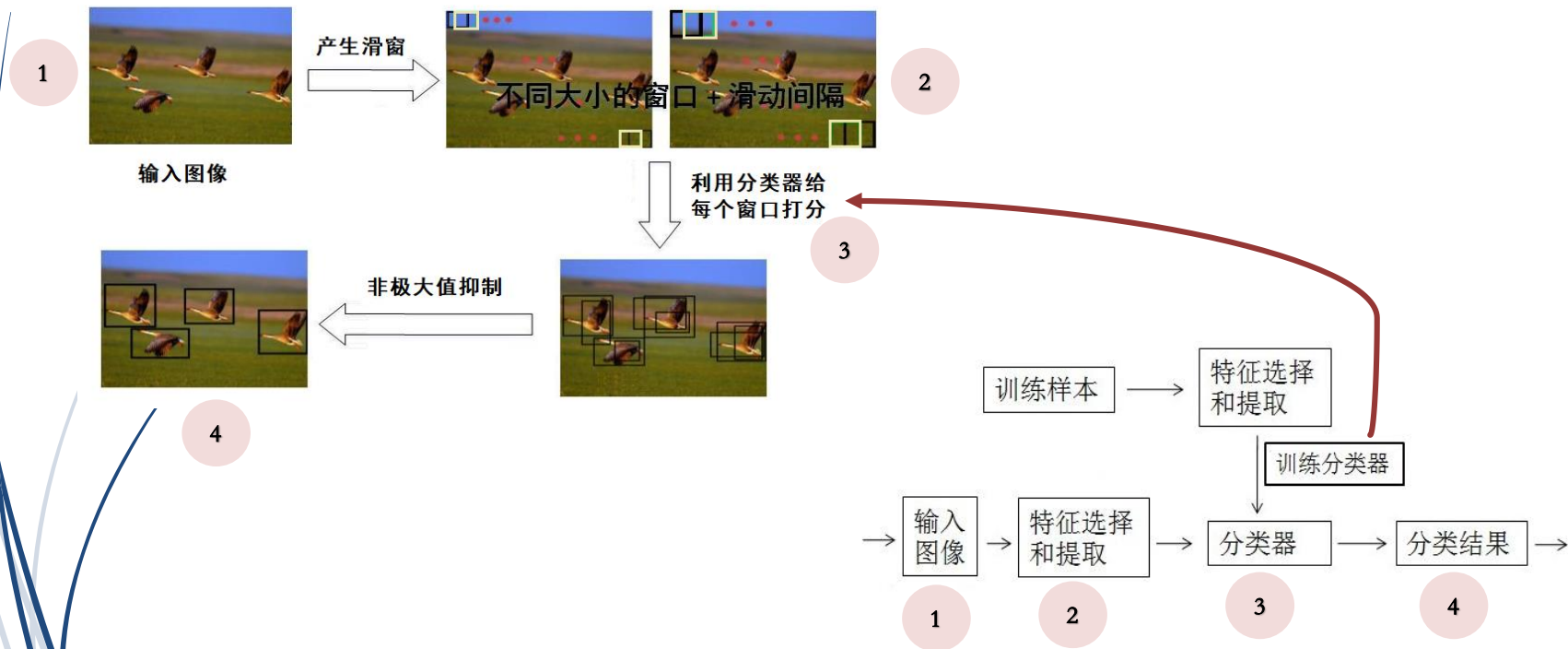
实例目标识别

滑动窗口的目标检测

- ◆ 在复杂背景下，通过滑动窗口搜索感兴趣的物体



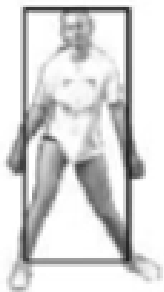
传统滑动窗口的物体检测实例



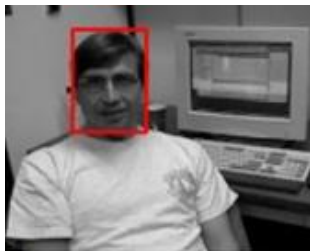
利用预先训练的分类器进行分类

结果表达方法

- ◆ 目标表达：能将目标从背景中分离出来
- ◆ 物体近似表达方法：



矩形包围盒

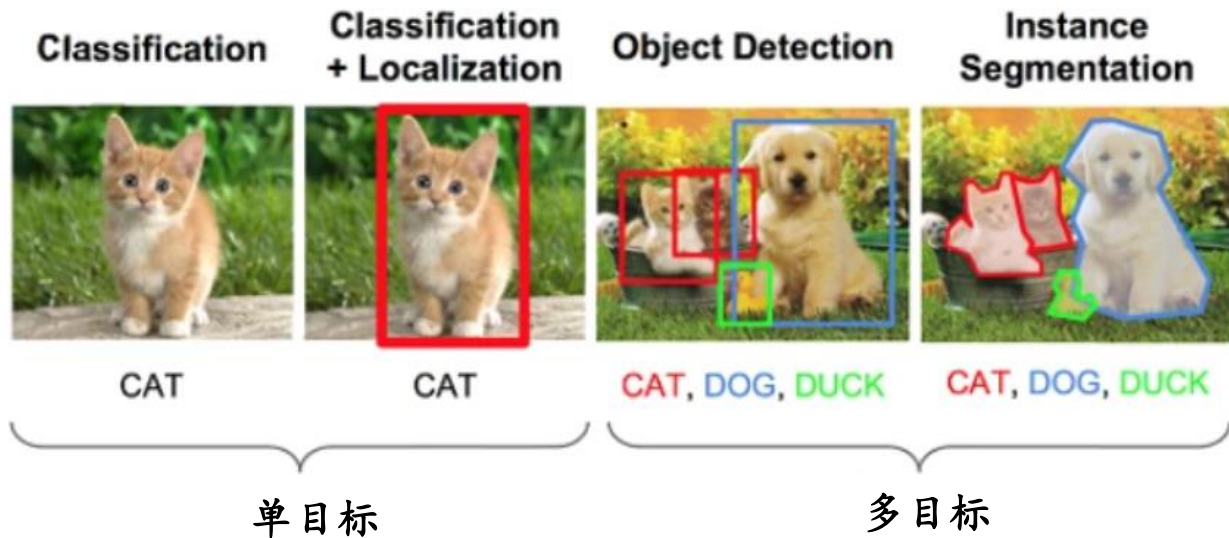


仅仅表示位置



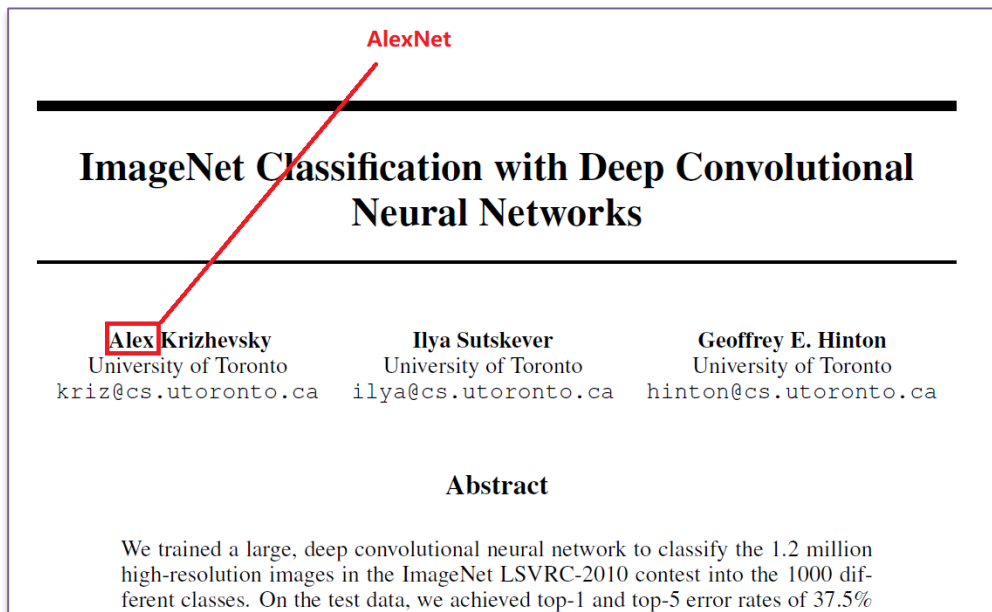
分割出区域

计算机视觉任务



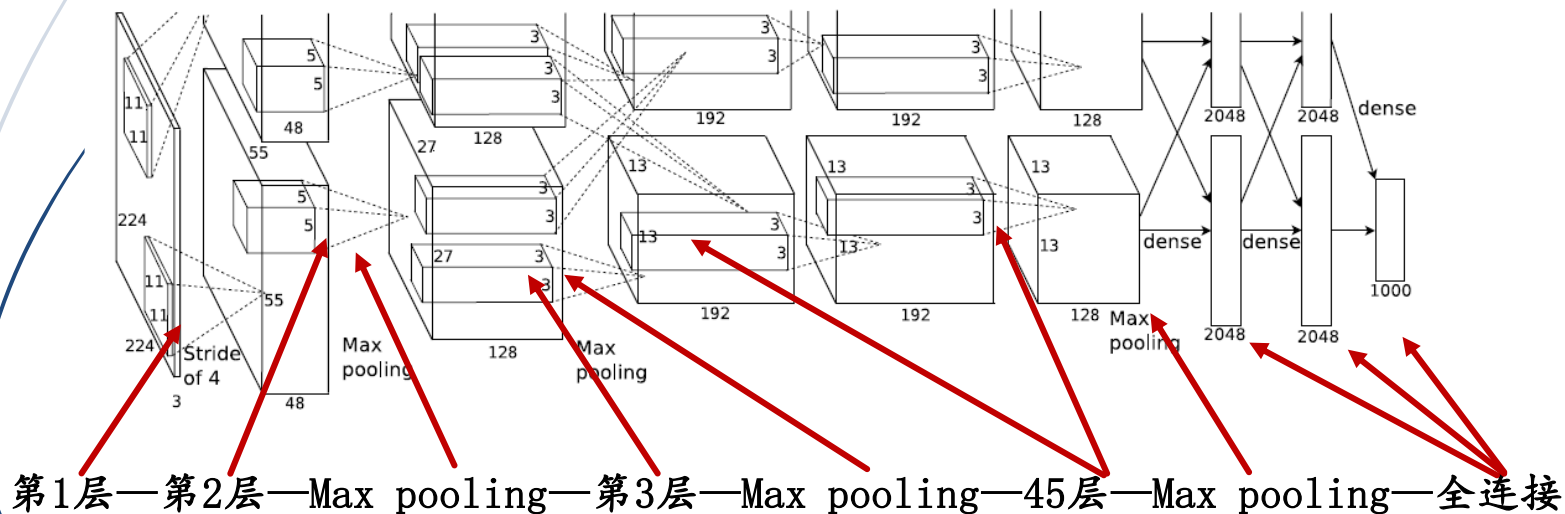
(2) 图像分类知识回顾

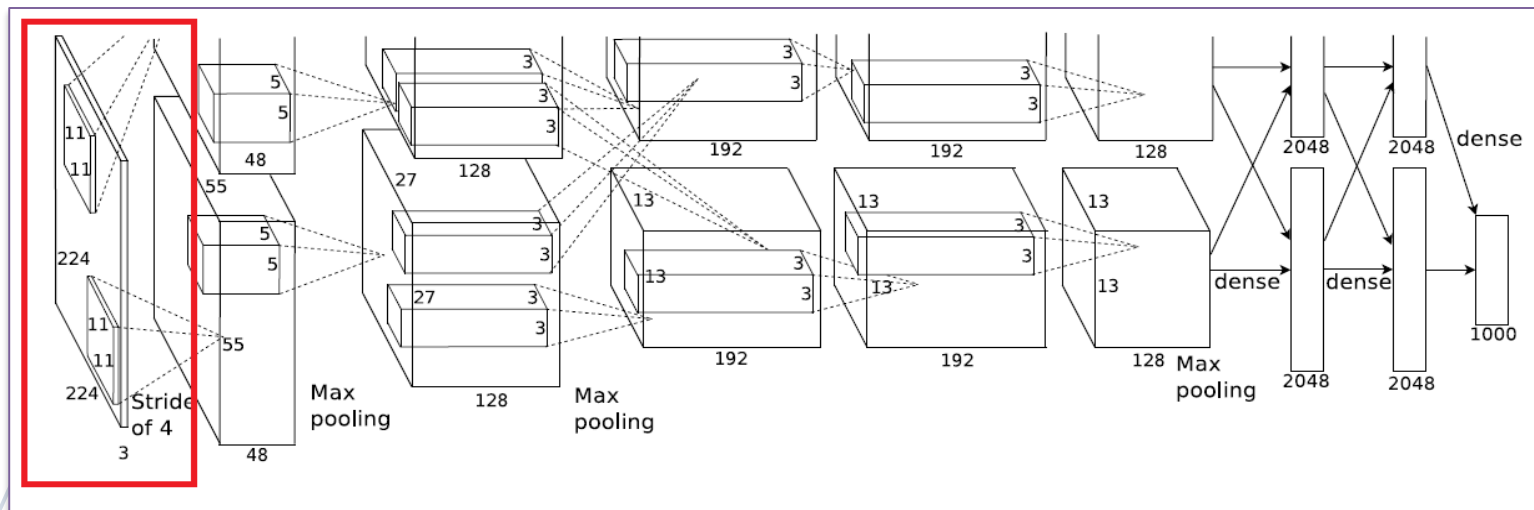
◆ AlexNet



AlexNet工作的主要原理

- ◆ AlexNet 以准确率84.6%赢得了ILSVRC-2012，由Krizhevsky等人提出。
- ◆ AlexNet包含5个卷积层，3个全连接层，结构如下图所示：

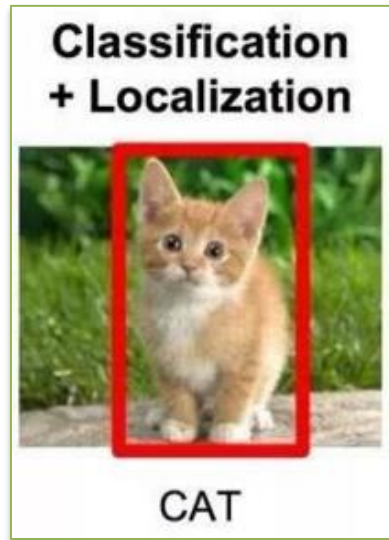




- ◆ 第一层为输入层，就是一幅224X224X3的image， 第一个卷积层， kernel 是11X11X3
- ◆ 每个11X11X3的图像区域卷积操作成为一个新的像素点的一个信道。卷积的步长(stride)为4， 说明224 X 224 被卷积成为是 55X55 的大小

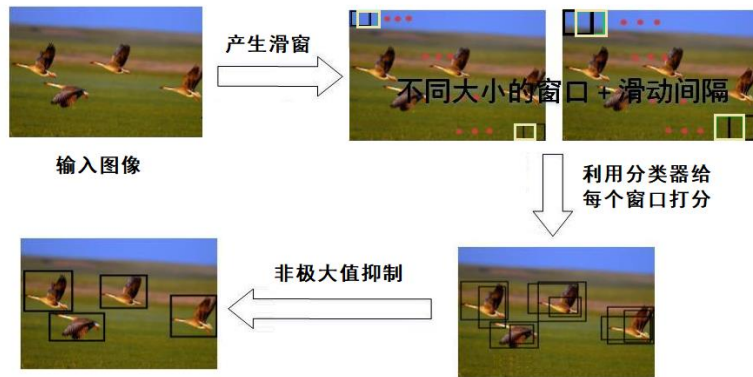
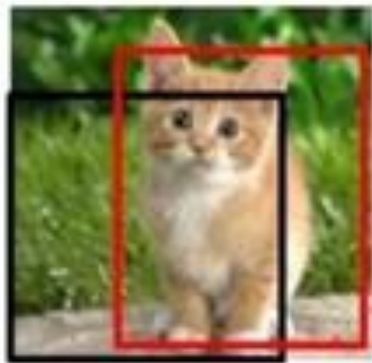
2. 目标识别问题

- ◆ 目标识别问题本质：分类与定位
 - 识别：分类（滑动框内对象）
 - 定位：预测出 (x, y, w, h) 四个参数的值
 - 本质问题：多任务
 - **任务1**：框内对象分类
 - **任务2**：框位置（四个参数，起始点，宽和高）



定位问题（位于哪里？）

- ◆ 思路来源：滑动窗口
- ◆ 定位框处于不同位置，然后打分判定



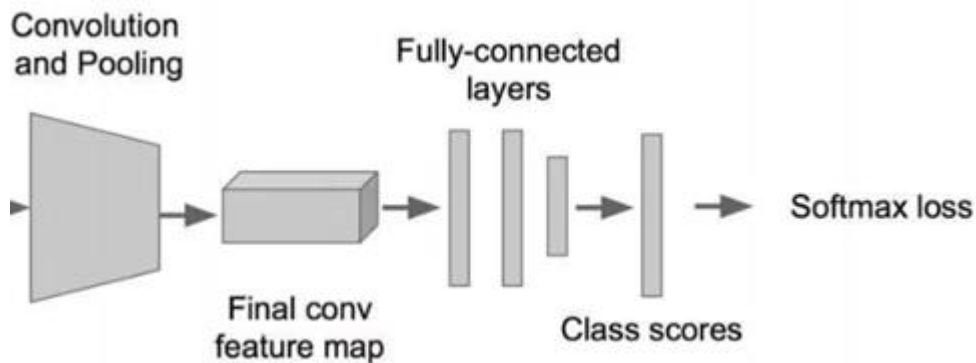
0.5	0.75
0.6	

传统方法的主要问题

- 1) 基于滑动窗口的区域选择策略没有针对性，时间复杂度高，窗口冗余
- 2) 对多实例分割较难实现

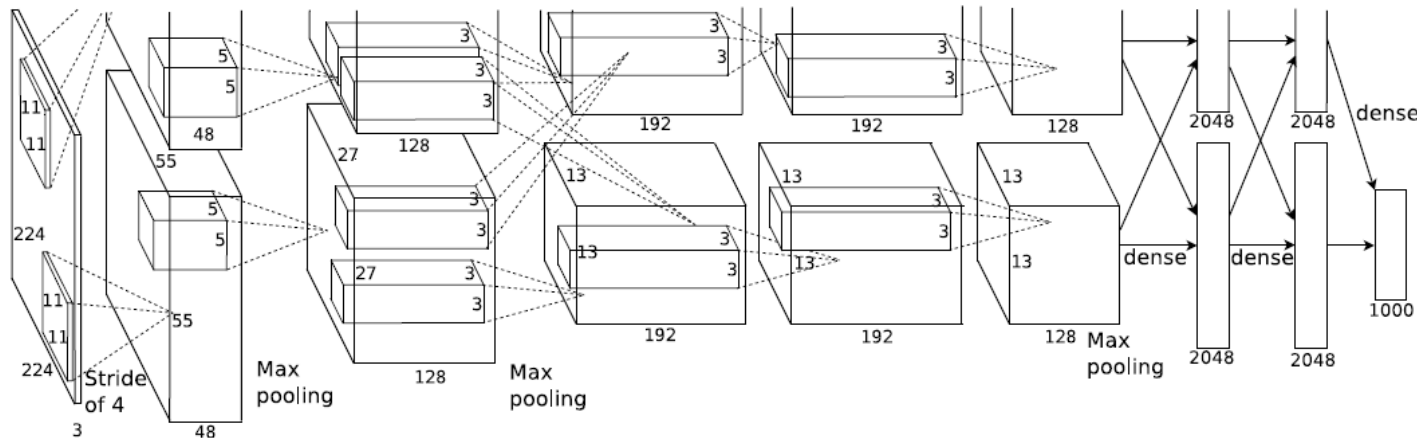
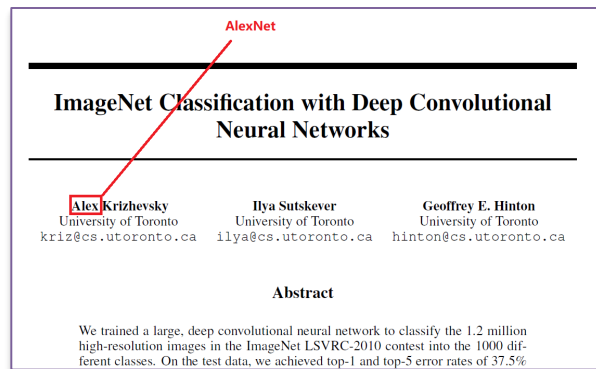
基于CNN的识别思路

- ◆ **方案1：** 利用现有网络改造
- ◆ **方案2：** 自己设计编码器，进行分类



方案1：利用现有网络改造

◆ 现有网络，例如AlexNet



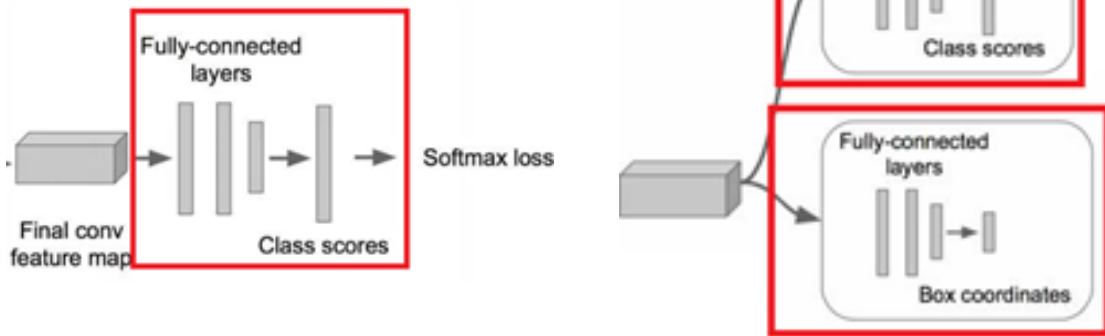
利用现有网络改造

◆ 目标识别问题本质：分类与定位

- 本质问题：多任务
 - **任务1**：框内对象分类
 - **任务2**：框位置（四个参数，起始点，宽和高）

Regression部分用欧氏距离损失

使用SGD (Stochastic Gradient Descent) 训练

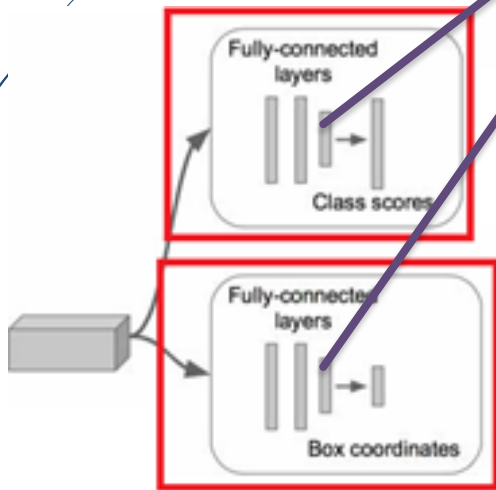


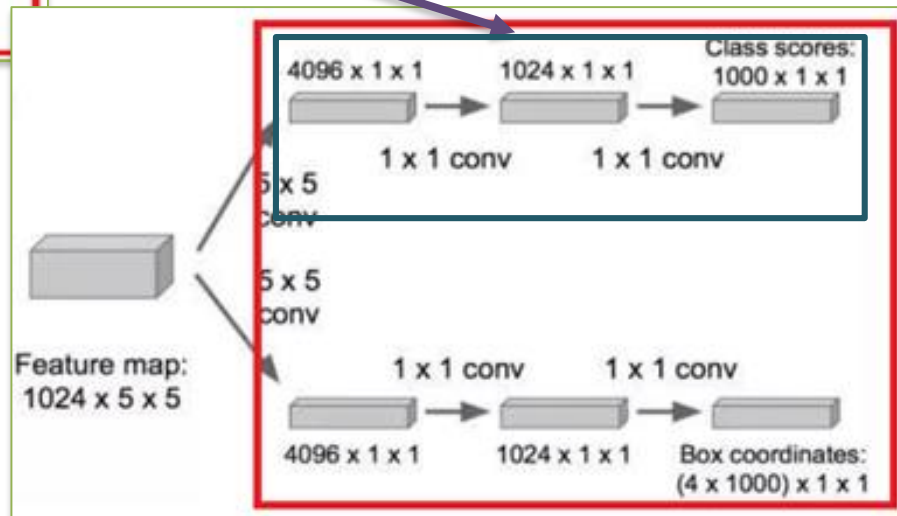
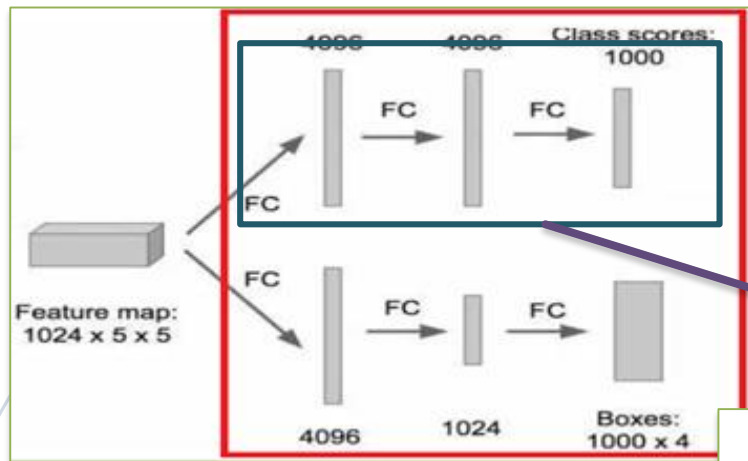
定位问题

- ◆ 输入：各种大小的框（遍历整张图片）将图片截取出来，输入到CNN
- ◆ 输出：网络regression，输出框的得分以及这个框图片对应的x, y, h, w

打分方法存在的问题

- ◆ 缺点：耗时间
- ◆ 解决方法：采用优化，全连接层改为卷积层，优化参数减少，提高速度





3. R-CNN系列算法

(1) R-CNN



This CVPR2014 paper is the Open Access version, provided by the Computer Vision Foundation.
The authoritative version of this paper is available in IEEE Xplore.

Rich feature hierarchies for accurate object detection and semantic segmentation

Ross Girshick¹ Jeff Donahue^{1,2} Trevor Darrell^{1,2} Jitendra Malik¹

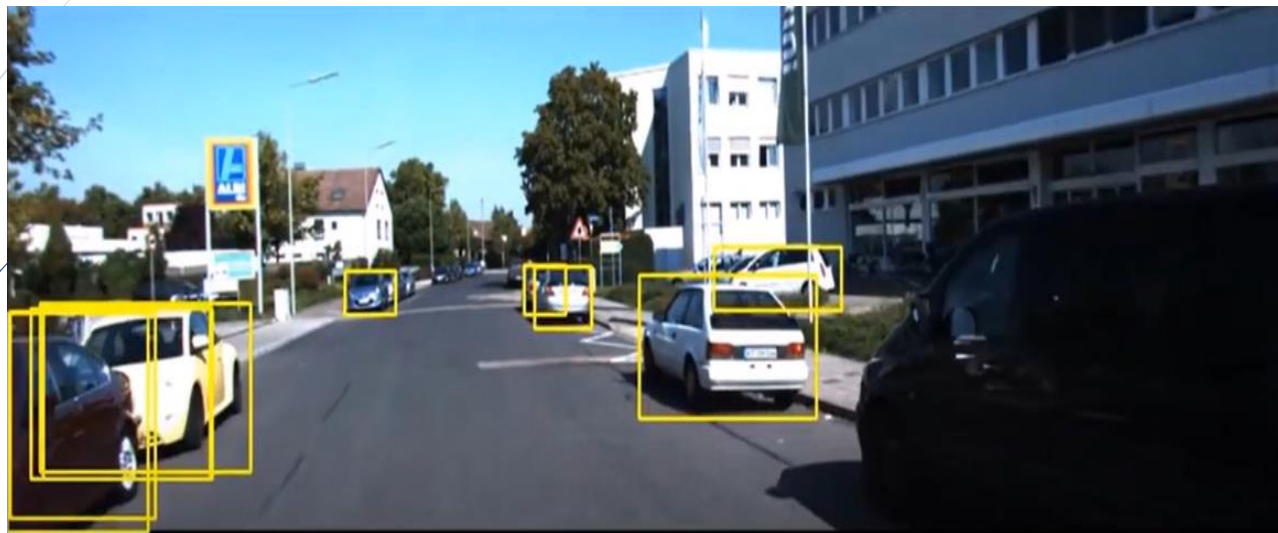
¹UC Berkeley and ²ICSI

{rbg, jdonahue, trevor, malik}@eecs.berkeley.edu

RCNN简介

- ◆ RCNN, 工业级应用的解决方案
- ◆ 2014年发布在顶级会议CVPR (Conference on Computer Vision and Pattern Recognition)

示例结果



R-CNN思路

- ◆ 候选窗结合深度学习方法
- ◆ 思路：
 - 先找出可能位置，即候选区域（Region Proposal）。
 - 再利用图像中的纹理、边缘、颜色等信息，提高准确性。

R-CNN: *Regions with CNN features*

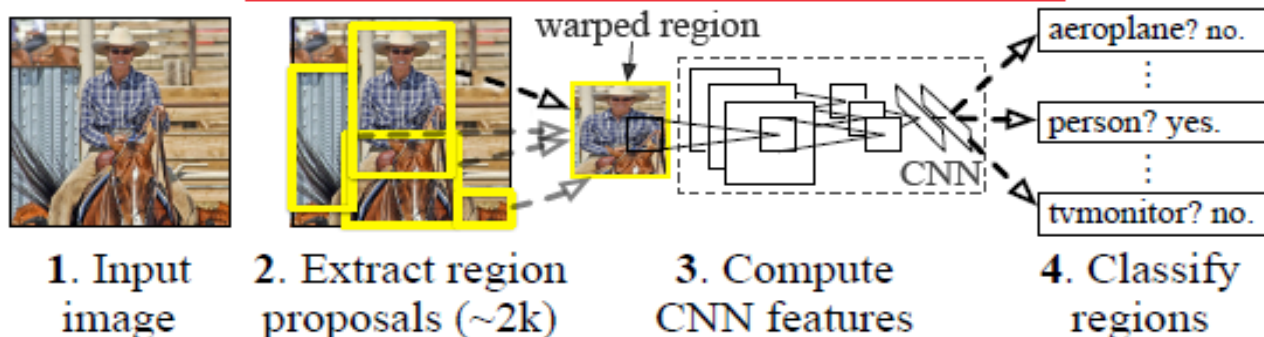


Figure 1: Object detection system overview. Our system (1) takes an input image, (2) extracts around 2000 bottom-up region proposals, (3) computes features for each proposal using a large convolutional neural network (CNN), and then (4) classifies each region using class-specific linear SVMs. R-CNN achieves a mean average precision (mAP) of **53.7% on PASCAL VOC 2010**. For comparison, [32] reports 35.1% mAP using the same region proposals, but with a spatial pyramid and bag-of-visual-words approach. The popular deformable part models perform at 33.4%.

R-CNN: *Regions with CNN features*

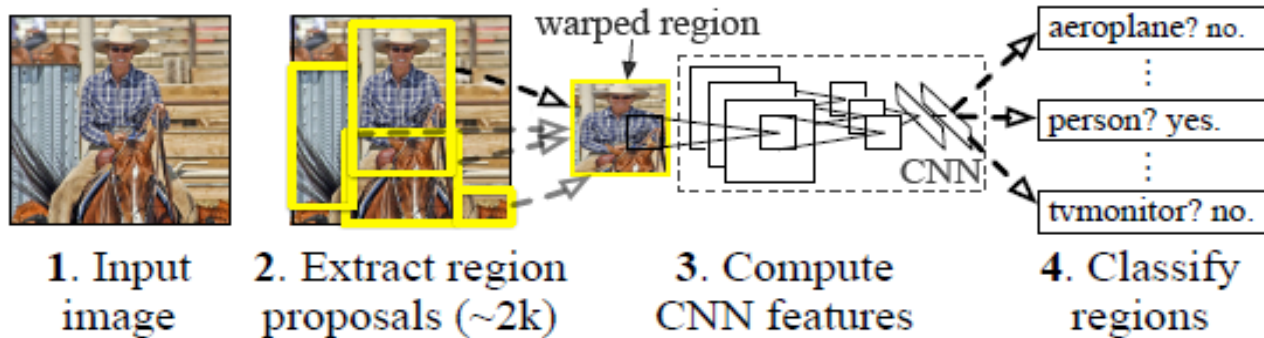


Figure 1: Object detection system overview. Our system (1) takes an input image, (2) extracts around 2000 bottom-up region proposals, (3) computes features for each proposal using a large convolutional neural network (CNN) region using class-specific linear SVM. In comparison, [32] reports 35.1% mAP on region proposals, but with a spatial pyramid pooling approach. The popular deformable part

Our object detection system consists of **three modules**. The first generates category-independent region proposals. These proposals define the set of candidate detections available to our detector. The second module is a large convolutional neural network that extracts a fixed-length feature vector from each region. The third module is a set of class-specific linear SVMs. In this section, we present our design

R-CNN的简要步骤如下

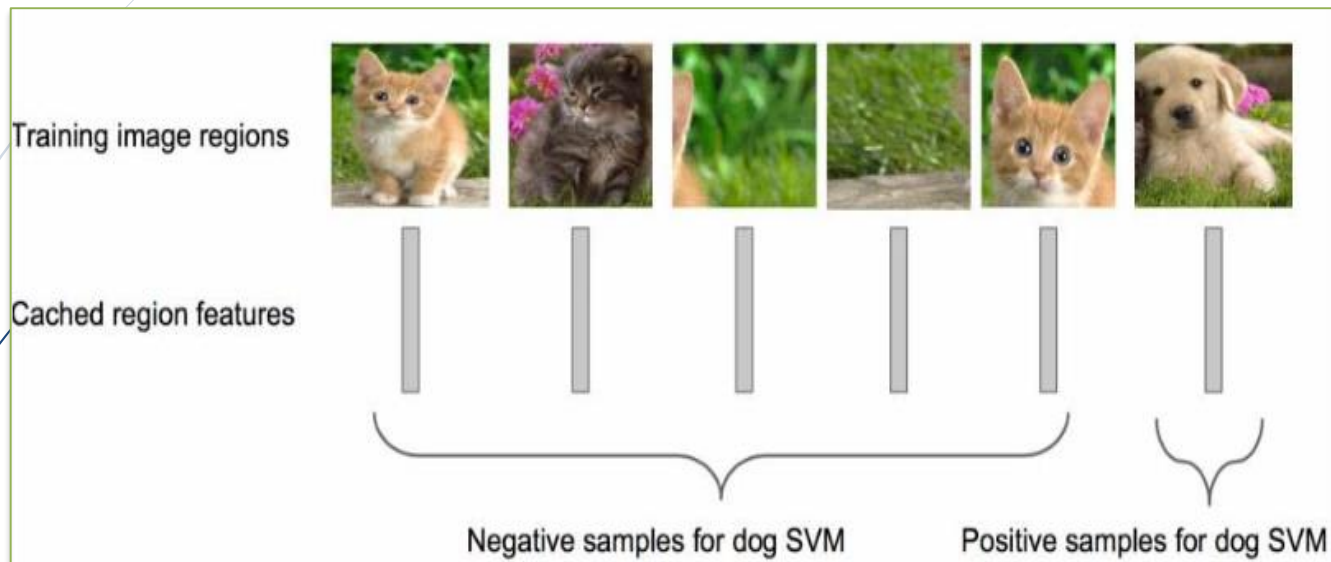
- (1) 在图像中提取2000个左右候选区域Region Proposal
- (2) 将每个Region Proposal缩放成统一的227x227大小并输入到CNN
- (3) 将CNN的fc7层的输出作为特征输入到SVM进行分类

选定候选框问题

- ◆ 出现的方法： Selective Search和EdgeBoxes
- ◆ 怎么选出这些候选框

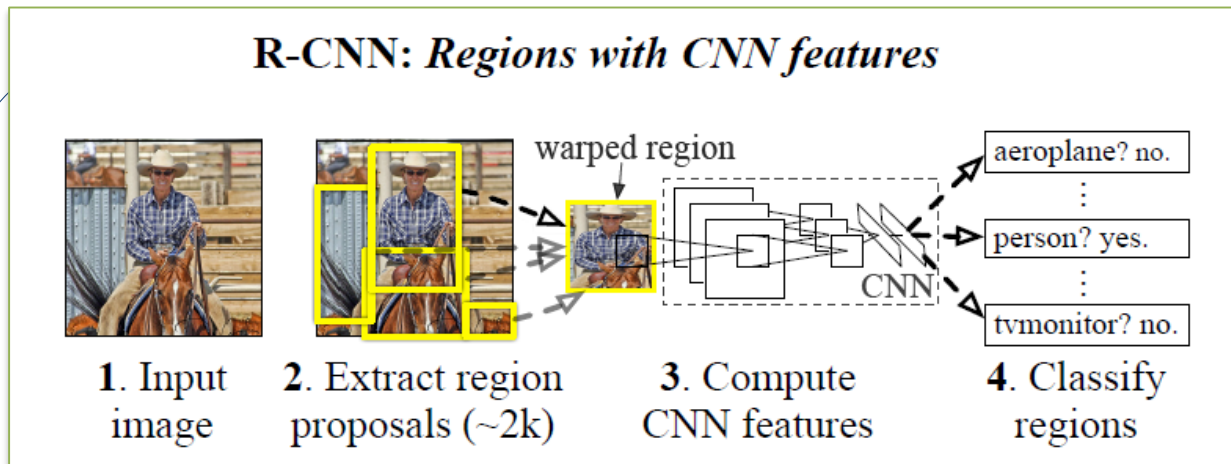
PAMI2015的 “What makes for effective detection proposals? ”

数据集



R-CNN存在的问题

- ◆ 候选框region proposal太多，每个框都需要进行CNN提特征+SVM分类，计算量很大，导致R-CNN检测速度很慢



(2) SPP Net

- ◆ SPP: Spatial Pyramid Pooling (空间金字塔池化)
- ◆ 2015年发表在IEEE上的论文:

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun: Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. IEEE Trans. Pattern Anal. Mach. Intell. 37(9): 1904–1916 (2015)

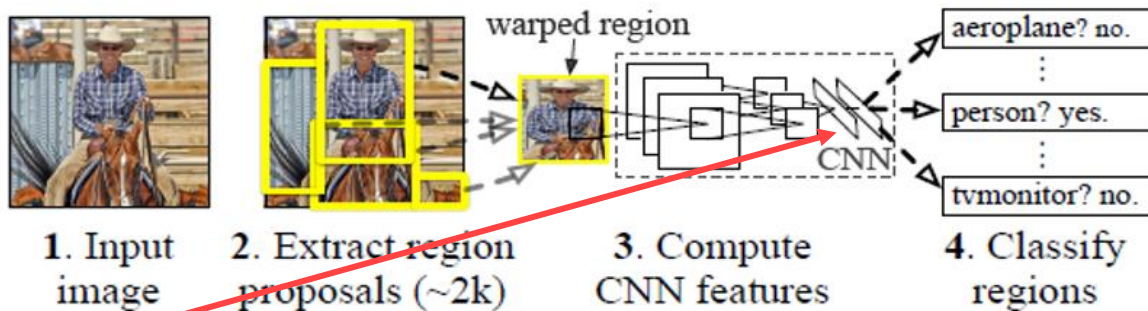
Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun

Abstract—Existing deep convolutional neural networks (CNNs) require a fixed-size (e.g., 224×224) input image. This requirement is “artificial” and may reduce the recognition accuracy for the images or sub-images of an arbitrary size/scale. In this work, we equip the networks with a more principled pooling strategy, “spatial pyramid pooling”, to eliminate the above requirement. The new network structure, called SPP net, can generate a fixed-length representation regardless of image size/scale. Pyramid

SPP Net的创新思路

- ◆ 鉴于现有算法存在的问题：

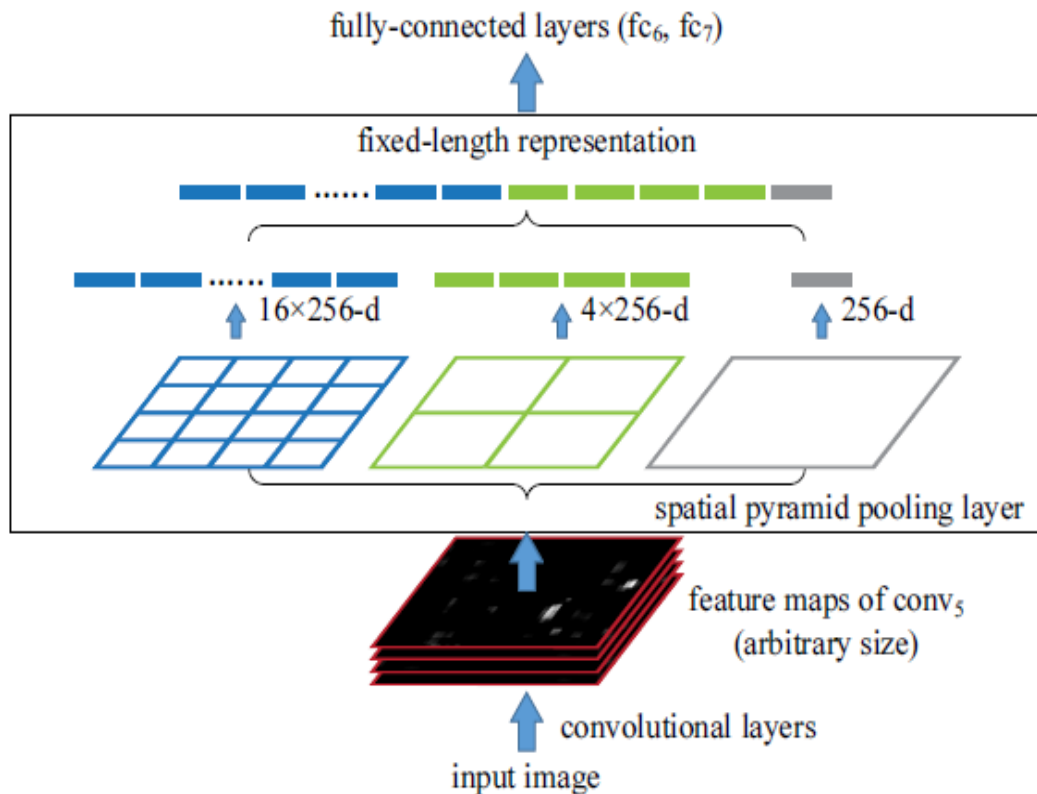


- ◆ 由于**全连接层存在**，对各种不同尺寸的图像框输入数据时，就需要对输入数据进行截取大小为固定大小的patch块，比如 227×227
- ◆ SPP Net在卷积层的最后加入了一个设计结构，使得卷积后的特征尺度变为固定大小，使得后面全连接层输入的特征尺度可以不变。

SPP Net的创新

- ◆ SPP Net的贡献是在最后一个卷积层后，接入了金字塔池化层，保证传到下一层全连接层的输入固定。
- ◆ ROI池化层一般跟在卷积层后面，此时网络的输入可以是任意尺度的，在SPP layer中每一个pooling的filter会根据输入调整大小。

SPP Net结构



不同方法对比

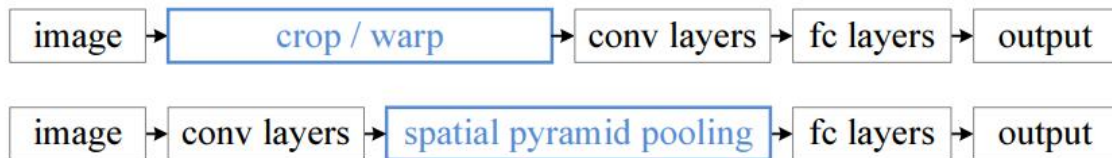


Figure 1: Top: cropping or warping to fit a fixed size. Middle: a conventional deep convolutional network structure. Bottom: our spatial pyramid pooling network structure.

SPP Net

- ◆ SPP Net只对原图提取一次卷积特征
- ◆ R-CNN要对每个区域计算卷积，而SPPNet只需要计算一次卷积，从而节省了大量的计算时间，比R-CNN速度改进很多倍。

model	conv ₁	conv ₂	conv ₃	conv ₄	conv ₅	conv ₆	conv ₇
ZF-5	96×7^2 , str 2 LRN, pool 3^2 , str 2 map size 55×55	256×5^2 , str 2 LRN, pool 3^2 , str 2 27×27	384×3^2 13×13	384×3^2 13×13	256×3^2 13×13	-	-
Convnet*-5	96×11^2 , str 4 LRN, map size 55×55	256×5^2 LRN, pool 3^2 , str 2 27×27	384×3^2 pool 3^2 , 2 13×13	384×3^2 13×13	256×3^2 13×13	-	-
Overfeat-5/7	96×7^2 , str 2 pool 3^2 , str 3, LRN map size 36×36	256×5^2 pool 2^2 , str 2 18×18	512×3^2 18×18	512×3^2 18×18	512×3^2 18×18	512×3^2 18×18	512×3^2 18×18

Table 1: Network architectures: filter number $\times$ filter size (e.g., 96×7^2), filter stride (e.g., str 2), pooling window size (e.g., pool 3^2), and the output feature map size (e.g., map size 55×55). LRN represents Local Response Normalization. The padding is adjusted to produce the expected output feature map size.

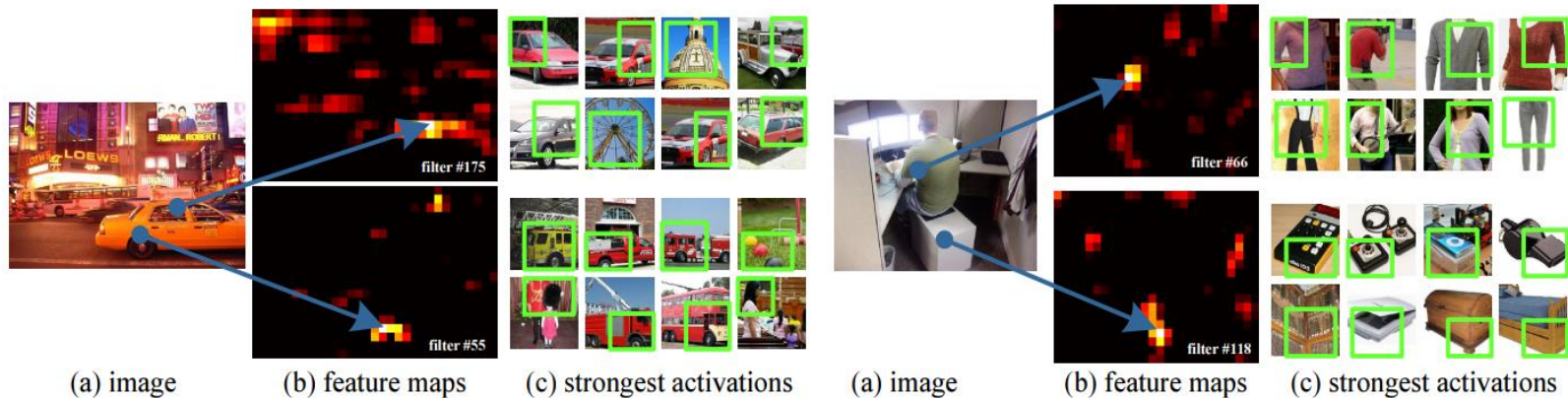


Figure 2: Visualization of the feature maps. (a) Two images in Pascal VOC 2007. (b) The feature maps of some conv₅ filters. The arrows indicate the strongest responses and their corresponding positions in the images. (c) The ImageNet images that have the strongest responses of the corresponding filters. The green rectangles mark the receptive fields of the strongest responses.

(3) Fast R-CNN

- ◆ Fast R-CNN是在R-CNN基础上采纳了SPP Net方法
- ◆ 对R-CNN作了改进，使得性能进一步提高

Fast R-CNN 结构

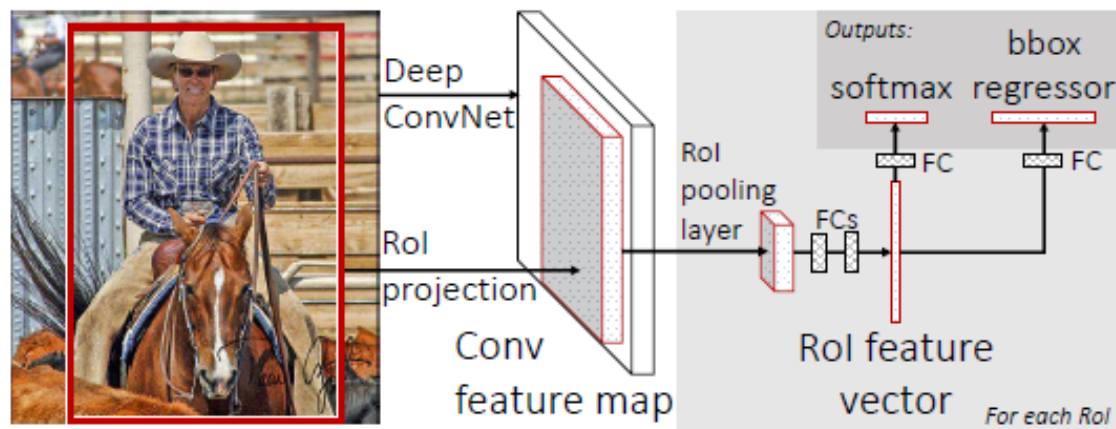


Figure 1. Fast R-CNN architecture. An input image and multiple regions of interest (RoIs) are input into a fully convolutional network. Each RoI is pooled into a fixed-size feature map and then mapped to a feature vector by fully connected layers (FCs). The network has two output vectors per RoI: softmax probabilities and per-class bounding-box regression offsets. The architecture is trained end-to-end with a multi-task loss.

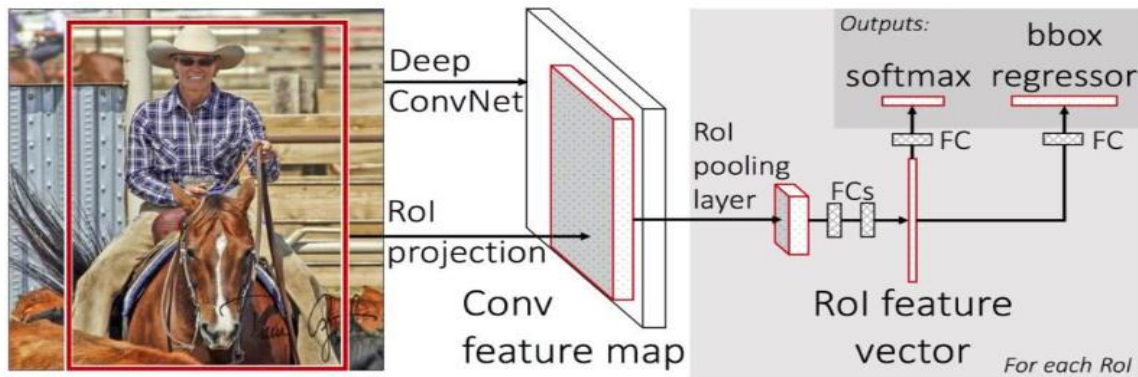
与R-CNN框架区别

- ◆ 与R-CNN框架图对比，主要有两处不同：
 - 最后一个卷积层后加了一个ROI pooling layer
 - 损失函数使用了多任务损失函数 (multi-task loss)，将边框回归Bounding Box Regression直接加入到CNN网络中训练

R-CNN训练过程

Fast R-CNN在R-CNN 基础上:

- ◆ 直接使用softmax替代SVM分类, 除了region proposal外, 采用端到端的训练过程, 将多任务损失函数边框回归加入到网络控制中。



Fast-RCNN贡献

- ◆ 成功设计Region Proposal结合CNN框架
- ◆ 实时检测位置，在多类检测同时，保证准确率，提升处理速度
- ◆ 为Faster R-CNN研究奠定基础

Fast R-CNN克服R-CNN缺点

- ◆ R-CNN的每一个候选框要独自经过CNN，花费时间
- ◆ Fast R-CNN不是每一个候选框输入进入CNN，而是输入一张完整的图片，再得到每个候选框的特征（第五个卷积层）

(4) Faster R-CNN

Faster R-CNN的提出：

- ◆ 为了解决Fast R-CNN存在的问题：选择性搜索候选框，这个也非常耗时
- ◆ Faster R-CNN中高效求取候选框
- ◆ Faster R-CNN加入一个提取边缘的神经网络，用神经网络找到候选框

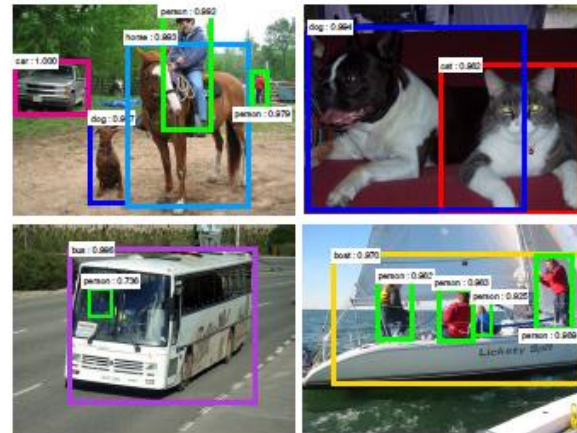
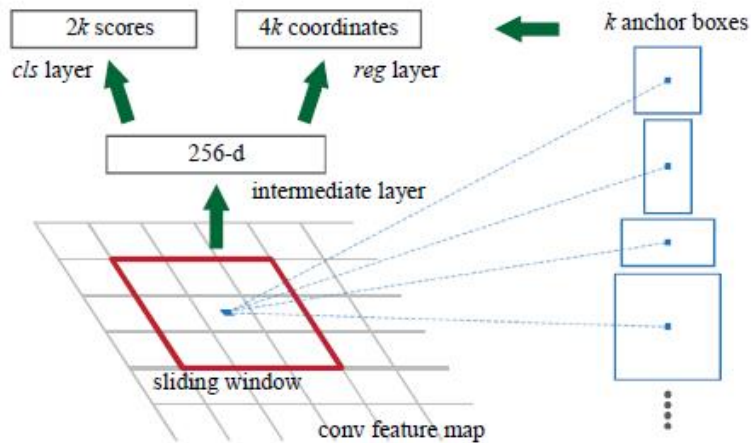
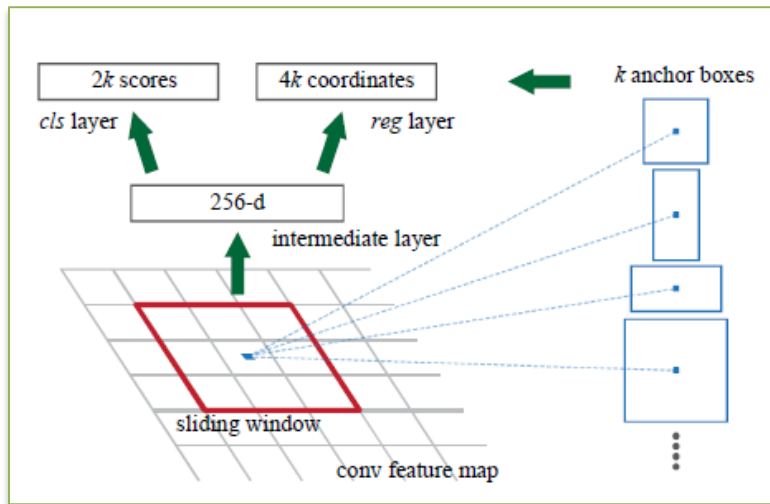


Figure 1: **Left:** Region Proposal Network (RPN). **Right:** Example detections using RPN proposals on PASCAL VOC 2007 test. Our method detects objects in a wide range of scales and aspect ratios.

RPN简介

- ◆ 在feature map上滑动窗口
- ◆ 建一个神经网络用于物体分类+框位置的回归
- ◆ 滑动窗口的位置提供了物体的大体位置信息
- ◆ 框的回归提供了框更精确的位置



Faster R-CNN的四个损失函数

- ◆ RPN classification(anchor good, bad)
- ◆ RPN regression(anchor $\rightarrow$ proposal)
- ◆ Fast R-CNN classification(over classes)
- ◆ Fast R-CNN regression(proposal $\rightarrow$ box)

Faster R-CNN主要贡献

- ◆ Faster R-CNN的主要贡献就是设计了提取候选区域的网络RPN，代替了费时的选择性搜索Selective Search，使得检测速度大幅提高。

R-CNN系列算法小结

- ◆ R-CNN (Selective Search + CNN + SVM)
- ◆ SPP-net (ROI Pooling)
- ◆ Fast R-CNN (Selective Search + CNN + ROI)
- ◆ Faster R-CNN (RPN + CNN + ROI)

4. YOLO系列算法

- ◆ 主要有两类目标检测方法

- R-CNN系列算法： two-stage，将目标检测任务分为边界框回归和物体分类两个模块
- YOLO系列算法： one-stage，将目标检测任务单纯看作回归任务。

- ◆ Faster R-CNN的方法曾经是主流的目标检测方法，但速度不能满足实时的要求
- ◆ YOLO系列算法使用了回归的思想，利用整张图作为网络的输入，直接在图像的多个位置上回归出这个位置的目标边框，以及目标所属的类别

- ◆ Faster R-CNN中尽管RPN与fast rcnn共享卷积层，但在模型训练过程中，需要反复训练RPN网络和fast rcnn网络，将检测结果分为两部分求解：物体类别（分类问题），物体位置即 bounding box（回归问题）
- ◆ YOLO不需求取region proposal过程，将目标识别统一为回归问题，将一幅图像分成若干个网格，使得YOLO可以每秒处理45张图像。如果某object的中心 落在这个网格中，则这个网格用于预测该object



(1) YOLOv1

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[¶], Ali Farhadi*[†]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[¶]

<http://pjreddie.com/yolo/>

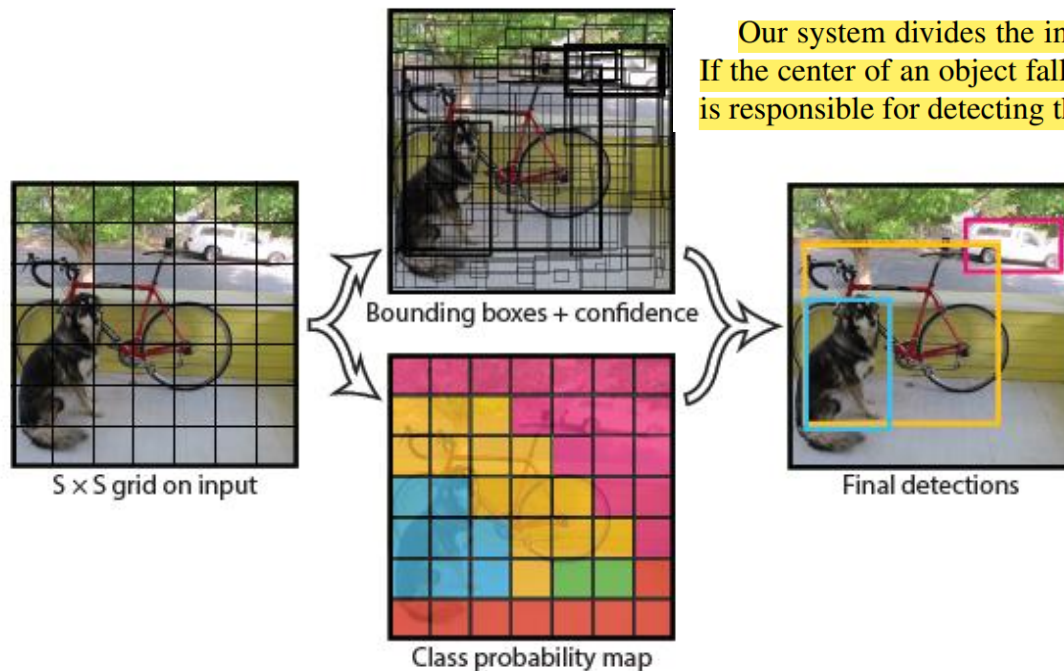


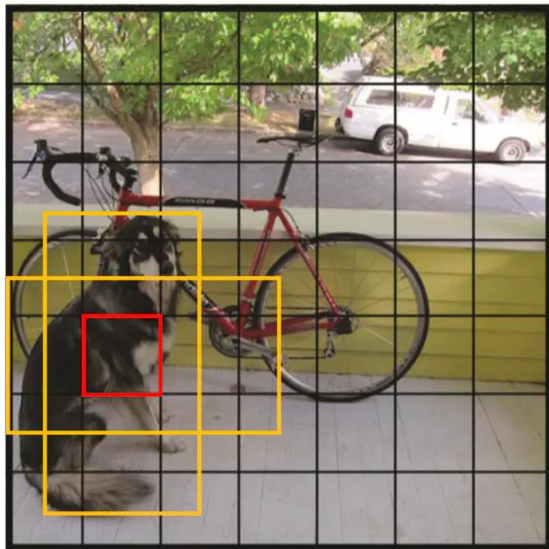
Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

算法步骤

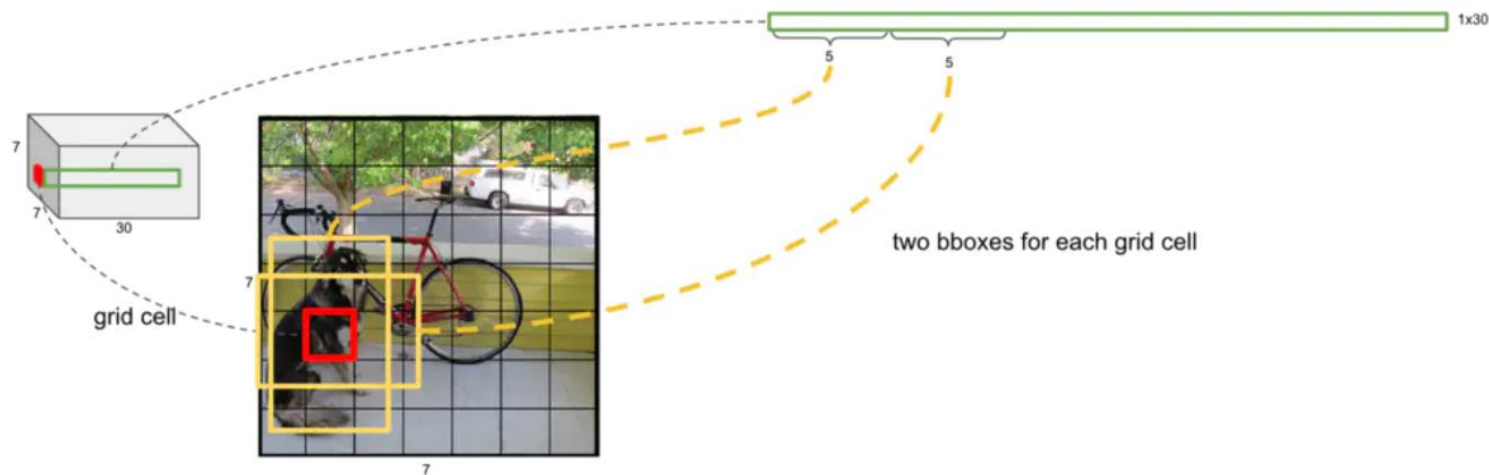
- (1) 对输入图像，划分成 $7*7$ 的若干网格
- (2) 对每个网格，预测2个边框
- (3) 可预测出 $7*7*2$ 个目标窗口，根据阈值去除可能性比较低的目标窗口，最后NMS去除冗余窗口即可

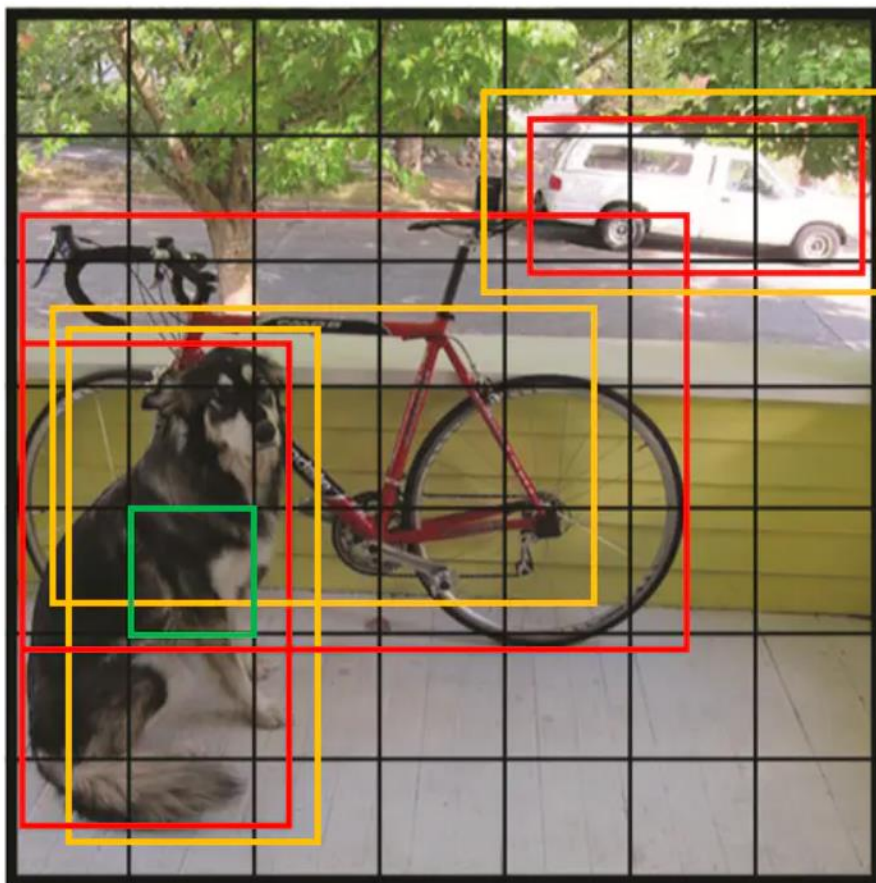
回归参数

- ◆ YOLOv1提出首先每个先验框有五个参数，分别是表示先验框中心坐标，表示先验框宽高以及表示锚框分数（包含物体概率大小）



- ◆ 输出的结果中，深度30表示每个网格需要预测30个参数（2个先验框共10个参数，还有需要预测的类别数20）。





算法网络结构

- ◆ 过程简单，不需要中间的region proposal找目标，直接回归完成位置和类别判定

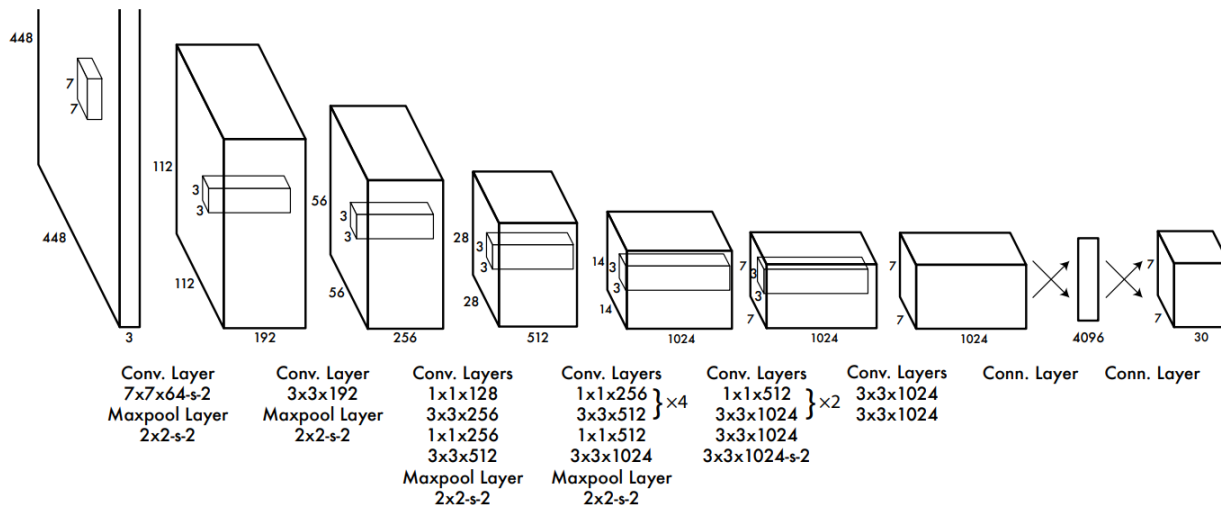


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

算法网络结构

- ◆ 过程简单，不需要中间的region proposal找目标，直接回归完成位置和类别判定

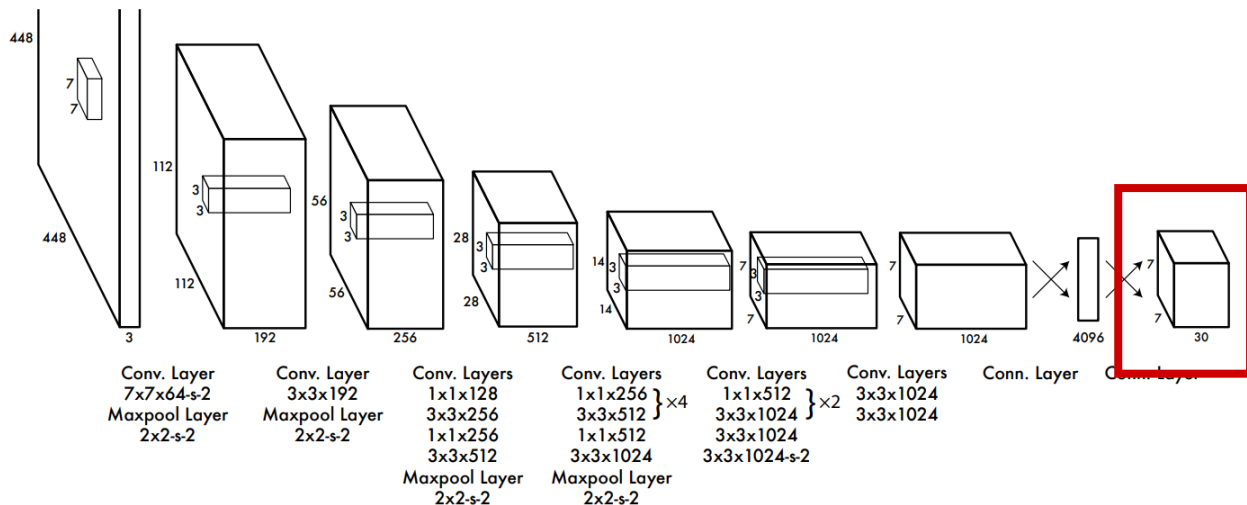


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

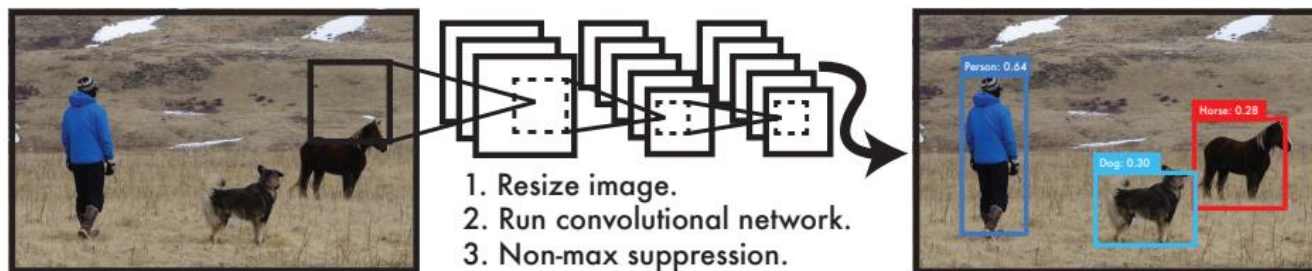


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

- ◆ 只有一条单一的网络分支，主干网： 由GoogLeNet网络修改
- ◆ 后面接一些全连接层
- ◆ YOLO将目标检测任务转换成回归问题，加快检测速度
- ◆ YOLO存在问题：
 - 没有了Region Proposal机制，只使用7*7的网格回归会使得目标不能非常精准的定位
 - 导致了YOLO的检测精度并不是很高

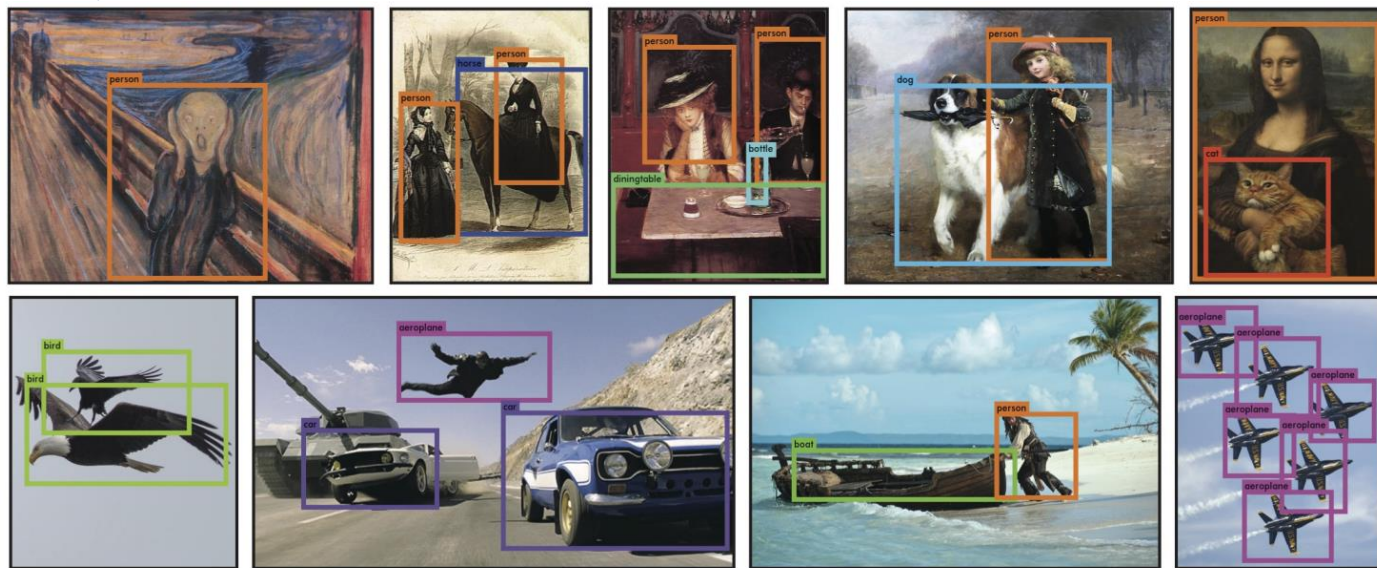
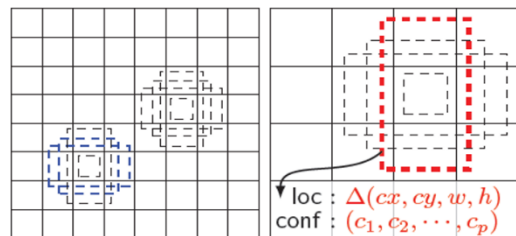


Figure 6: Qualitative Results. YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

(2) SSD (SSD: Single Shot MultiBox Detector)

- ◆ YOLO v1存在的问题，使用整图特征在7*7的粗糙网格内回归对目标的定位不是很精准
- ◆ SSD结合YOLO的回归思想以及Faster R-CNN的anchor机制，结合region proposal的思想实现较精准定位

- ◆ 使用各个位置的多尺度区域特征进行回归



(b) 8×8 feature map (c) 4×4 feature map

- ◆ SSD检测物体，预先抽象一组bounding box（尺度，长宽比）
- ◆ SSD在每个bounding box，预测分类label，以及box offset来更好的框出物体。

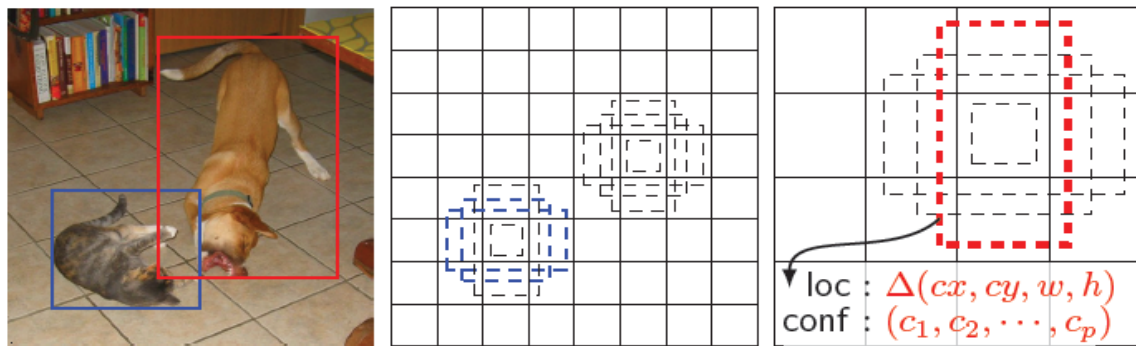
(2) SSD (SSD: Single Shot MultiBox Detector)

- ◆ Fu, Alexander C. Berg: SSD: Single Shot MultiBox Detector. ECCV, 2016: 21–37

SSD: Single Shot MultiBox Detector

Wei Liu¹, Dragomir Anguelov², Dumitru Erhan³, Christian Szegedy³,
Scott Reed⁴, Cheng-Yang Fu¹, Alexander C. Berg¹

¹UNC Chapel Hill ²Zoox Inc. ³Google Inc. ⁴University of Michigan, Ann-Arbor
¹wliu@cs.unc.edu, ²drago@zoox.com, ³{dumitru, szegedy}@google.com,
⁴reedscot@umich.edu, ¹{cyfu, aberg}@cs.unc.edu



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

Fig. 1: SSD framework. (a) SSD only needs an input image and ground truth boxes for each object during training. In a convolutional fashion, we evaluate a small set (e.g. 4) of default boxes of different aspect ratios at each location in several feature maps with different scales (e.g. 8×8 and 4×4 in (b) and (c)). For each default box, we predict both the shape offsets and the confidences for all object categories $((c_1, c_2, \dots, c_p))$. At training time, we first match these default boxes to the ground truth boxes. For example, we have matched two default boxes with the cat and one with the dog, which are treated as positives and the rest as negatives. The model loss is a weighted sum between localization loss (e.g. Smooth L1 [6]) and confidence loss (e.g. Softmax).

SSD设计的主要思想

- ◆ SSD结合了YOLO中的回归思想和Faster R-CNN中的anchor机制
- ◆ 使用各个位置的多尺度区域特征进行回归，保持了YOLO速度快的特性，也保留了Faster R-CNN的精准性

SSD算法原理

- ◆ 与YOLO一样，SSD获取目标位置和类别，但是YOLO预测某个位置使用的是全图的特征，SSD预测某个位置使用的是这个位置周围的特征
- ◆ 如何建立某个位置和其特征的对应关系
 - 某一层特征图大小是 $8*8$ ，使用 $3*3$ 的滑窗提取每个位置的特征，然后这个特征回归得到目标的坐标信息和类别信息。
 - feature map上，利用多层的特征，结合多尺度（不同层的feature map $3*3$ 滑窗感受野不同）

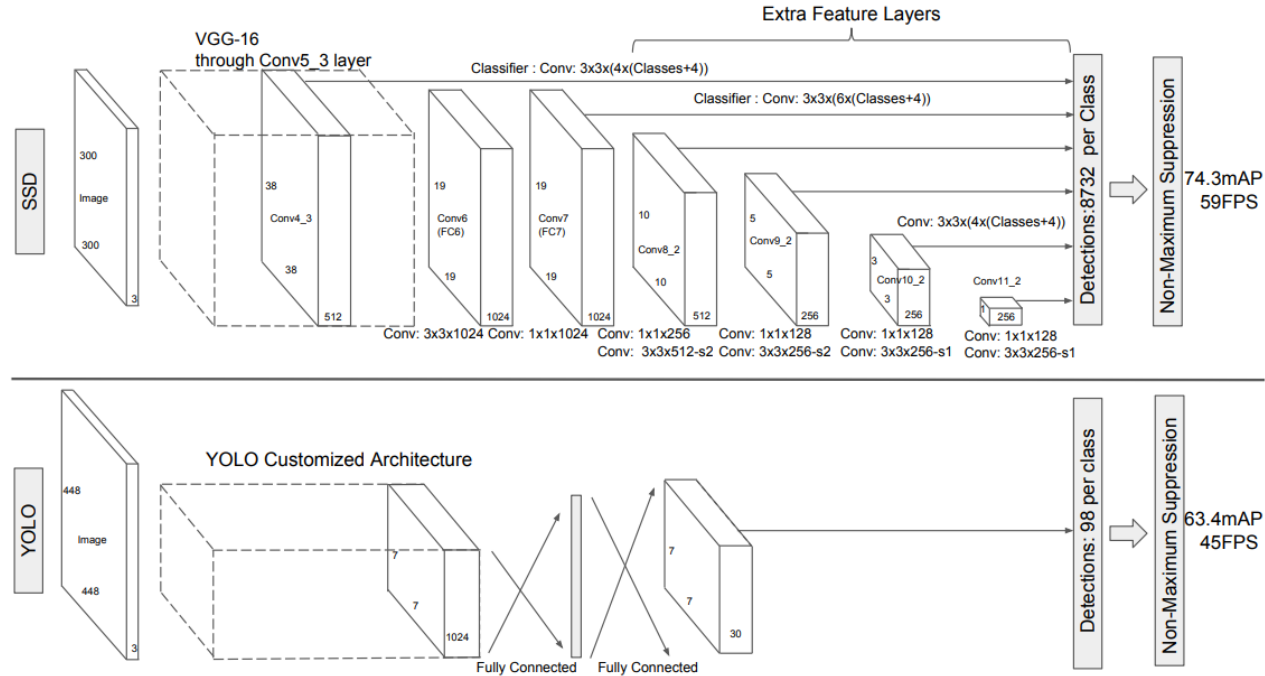


Fig. 2: A comparison between two single shot detection models: SSD and YOLO [5]. Our SSD model adds several feature layers to the end of a base network, which predict

(3) YOLO-V2

◆ 问题背景

- 问题1: YOLOv1 检测性能低
- 问题2: YOLOv1检测任务受数据集标签的限制

◆ YOLO-V2创新点

- 针对问题1 , 即YOLOv1 检测性能低问题, 提升 YOLOv1 的性能, 得到 YOLOv2
- 针对问题2 , 提出了 ImageNet 和 COCO 数据集的结合方法, 以及联合训练方法, 训练 YOLOv2 后得到的模型为 YOLO9000 (ImageNet1000种类+pascal VOC , 分为9000类)

YOLO2路线

- ◆ YOLO2借鉴Faster RCNN路线，尝试采用先验框（anchor）。
- ◆ 在每个grid预先设定一组不同大小和宽高比的边框，覆盖不同位置，并且多种尺度
- ◆ 先验框作为预定义的候选区，利用神经网络检测其中是否存在目标对象
- ◆ 采用微调边框的位置方法

YOLO-V2

- ◆ V2 特点:
 - 功能强，速度快
- ◆ BN: 加快模型收敛
- ◆ **Multi-scale training:** 相比v1, v2中无全连接层, 可接受任意大小的输入, 采取多尺度训练的方式, 训练的模型可接受任意大小的输入。
- ◆ YOLOv2和SSD精度接近, 但速度更快。

yolo v2/yolo9000

- ◆ 引入了anchor box 的思想
- ◆ 输出层：用卷积层替代了yolo v1 的全连接层
- ◆ 识别种类、精度、速度、定位准确性都有很大提高

- ◆ 将训练数据集中的矩形框的宽高，用kmeans聚类得到先验框的宽和高
- ◆ 采用聚类方法得到被选作中心的边框centroid，假设其它边框 为box
- ◆ 用标准的欧式距离度量其它边框的误差：

$$d(box, centroid) = 1 - IOU(box, centroid)$$

预测边框位置

- ◆ 中心点预测的是相对grid左上角的offset。 p_w 和 p_h 为先验框

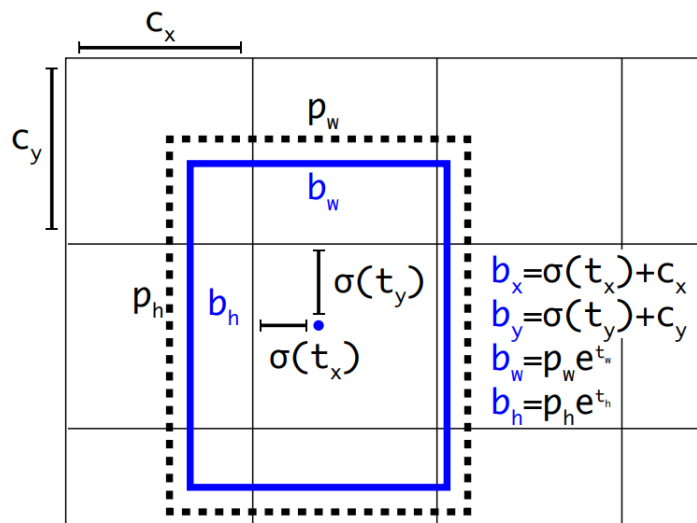
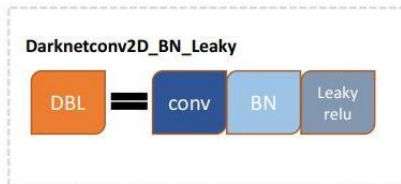
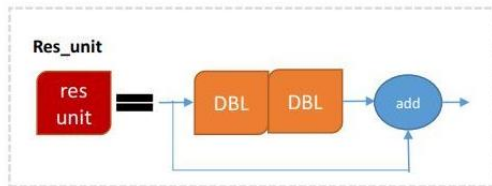
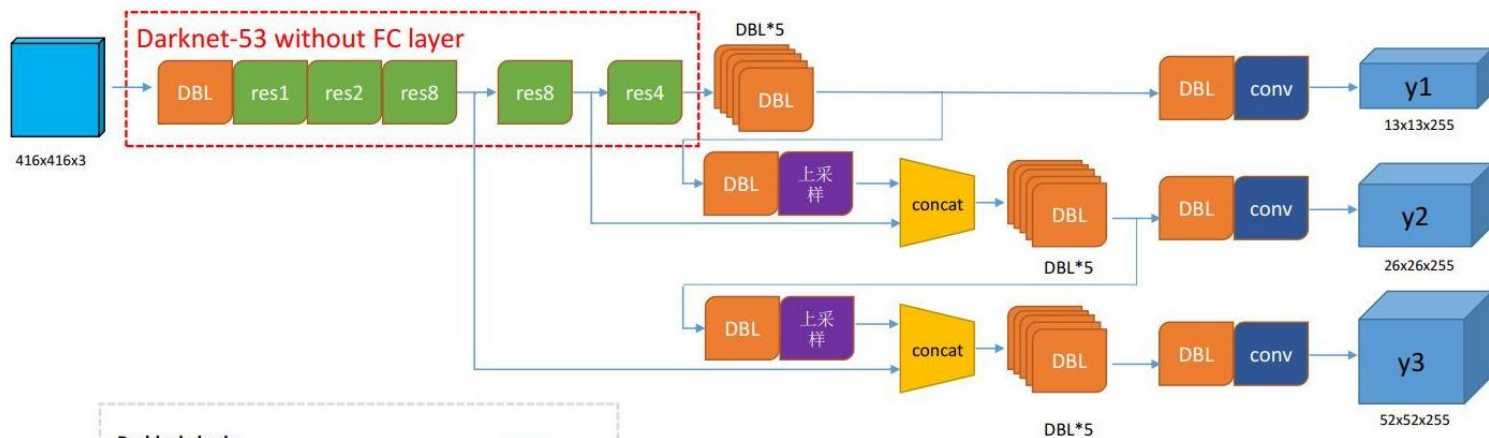


Figure 3: Bounding boxes with dimension priors and location prediction. We predict the width and height of the box as offsets from cluster centroids. We predict the center coordinates of the box relative to the location of filter application using a sigmoid function.

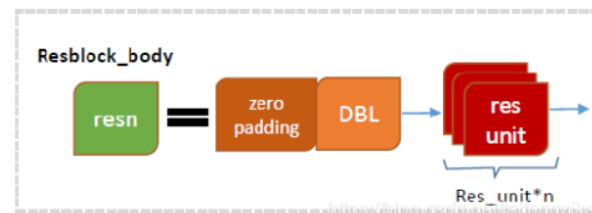
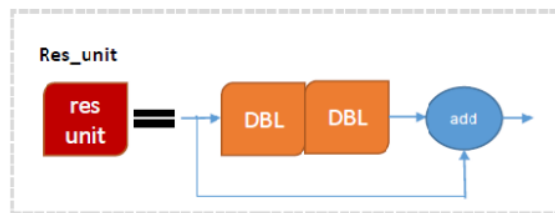
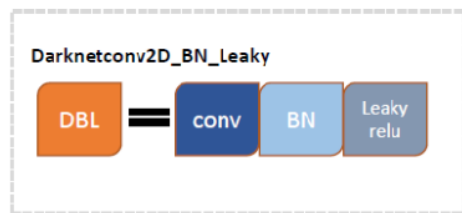
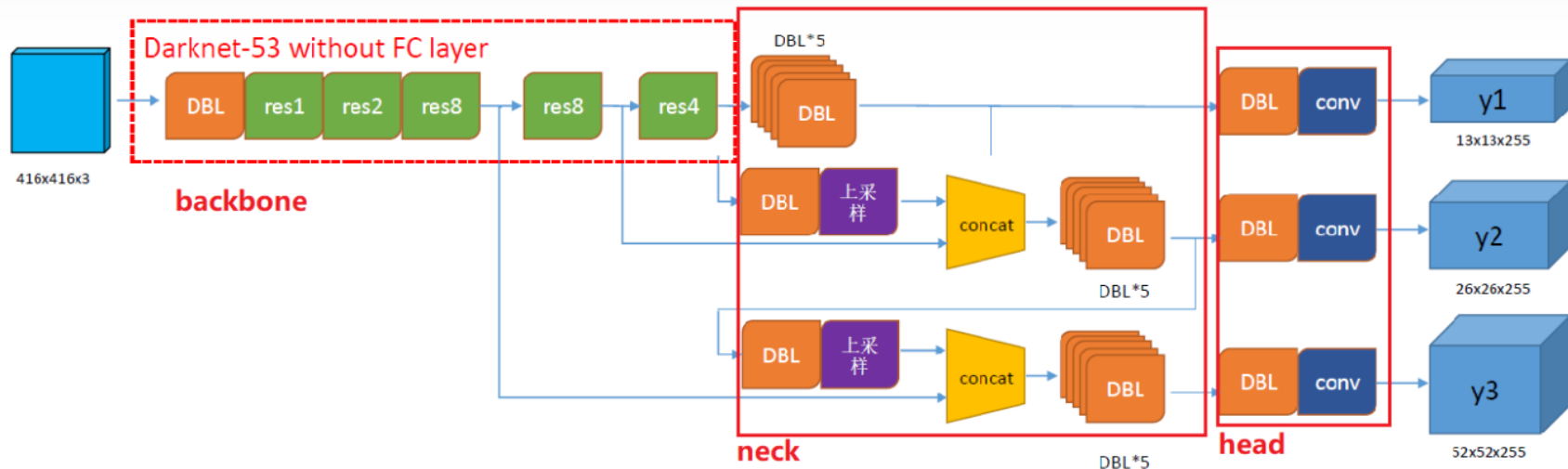
(4) YOLO-V3

- ◆ 基于v2结构和训练技巧进行改进，提升精度，包含以下核心改进：
 - 在多个不同分辨率特征图输出预测信息；结合浅层特征图的细节和深层特征语义信息
 - 中心点预测的是相对grid左上角的offset，这点和SSD不同，最终预框可更精准。
- ◆ 在yolo系列算法中，预测框包含物体置信度和分类类别置信度。

网络主体部分



Yolo_v3_Structure



网络说明

- ◆ DBL: yolo_v3的基本组件, 即卷积+BN+Leaky relu
- ◆ resn: 有res1, res2, ..., res8等等, 数字n代表这个res_block里含有多少个res_unit
- ◆ concat: 张量拼接。将darknet中间层和后面的某一层上采样特征进行拼接
- ◆ yolo v3输出了3个不同尺度的feature map: y1, y2, y3。
- ◆ predictions across scales借鉴了FPN(feature pyramid networks)多尺度思想
- ◆ 采用不同size的目标检测, 可以检测出越精细物体尺度

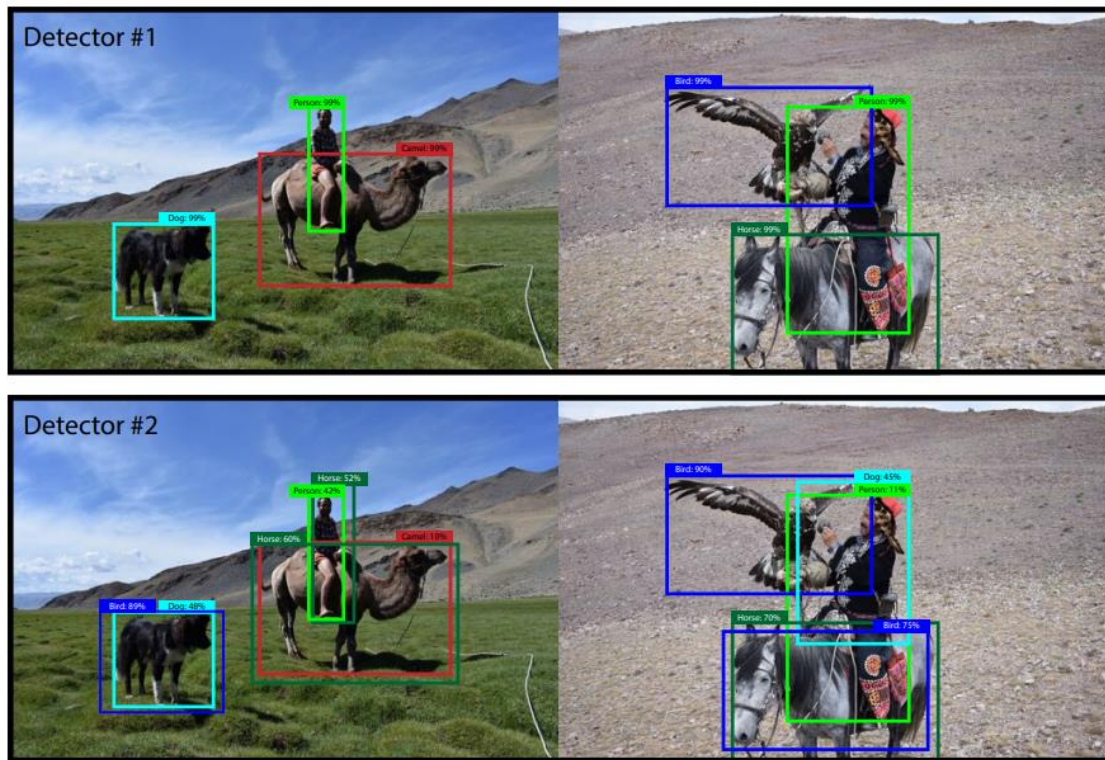
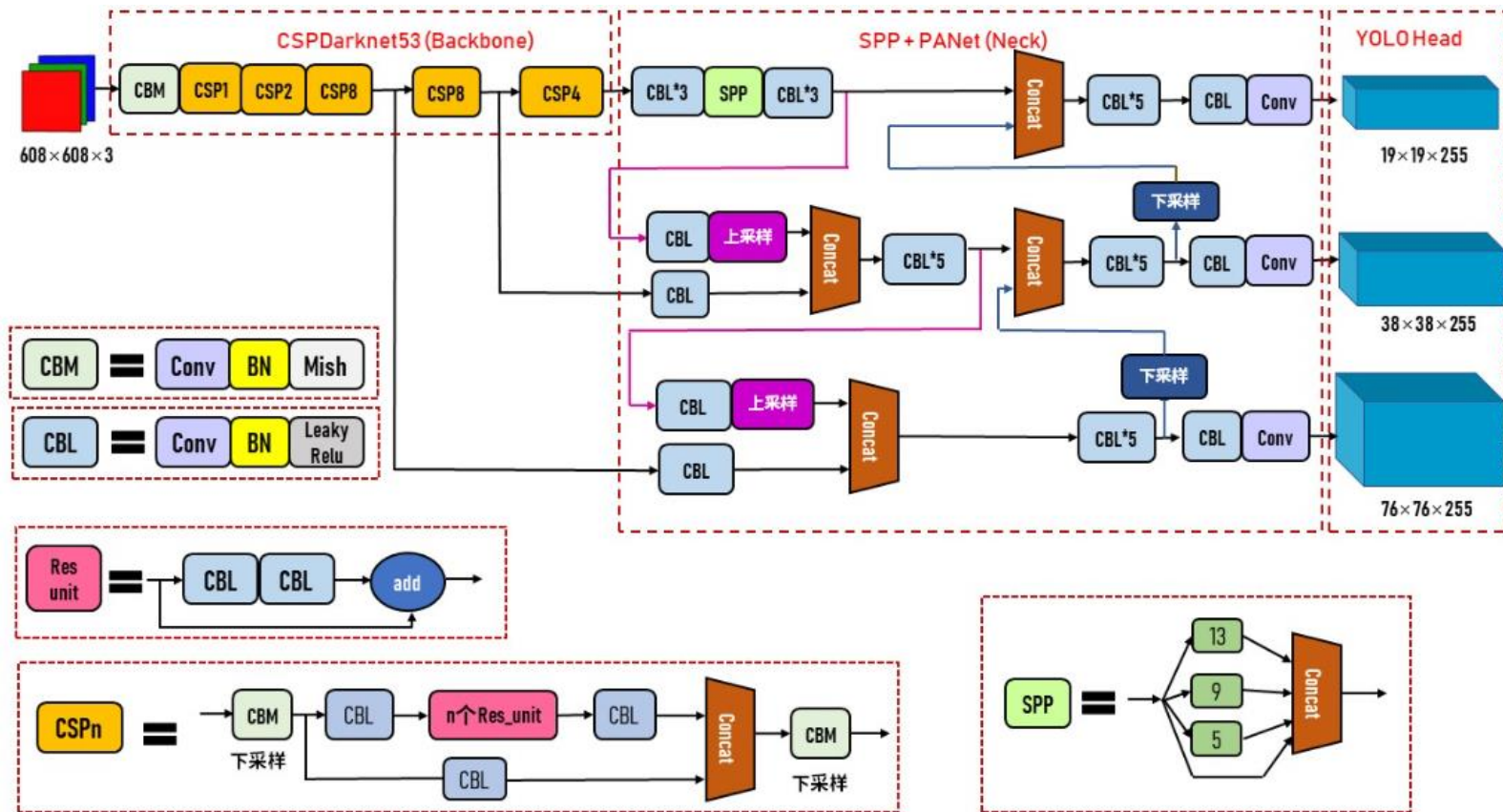


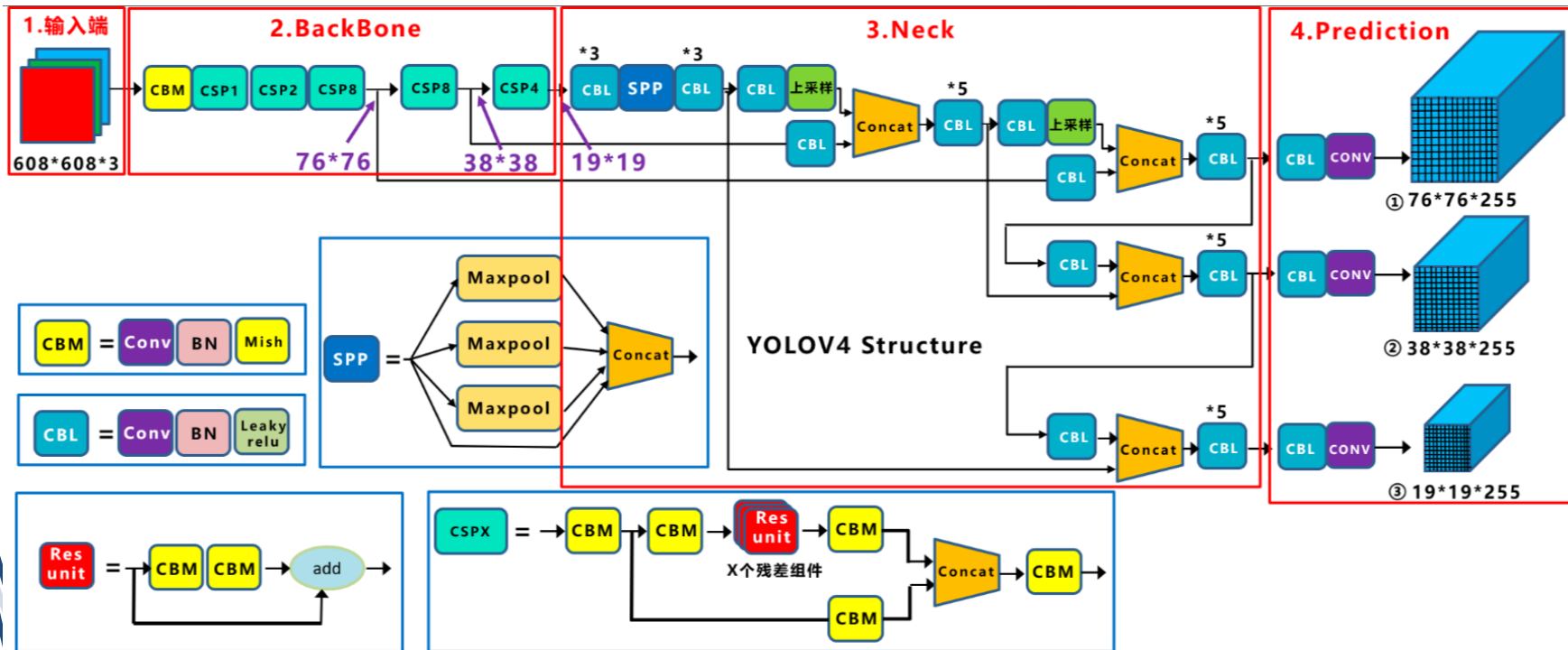
Figure 5. These two hypothetical detectors are perfect according to mAP over these two images. They are both perfect. Totally equal.

(5) YOLO-v4

- ◆ YOLO-v4算法YOLO目标检测架构的基础上，采用了优化策略
- ◆ 优化：
 - 主干网络
 - 网络训练
 - 激活函数
 - 损失函数

主要网络部分



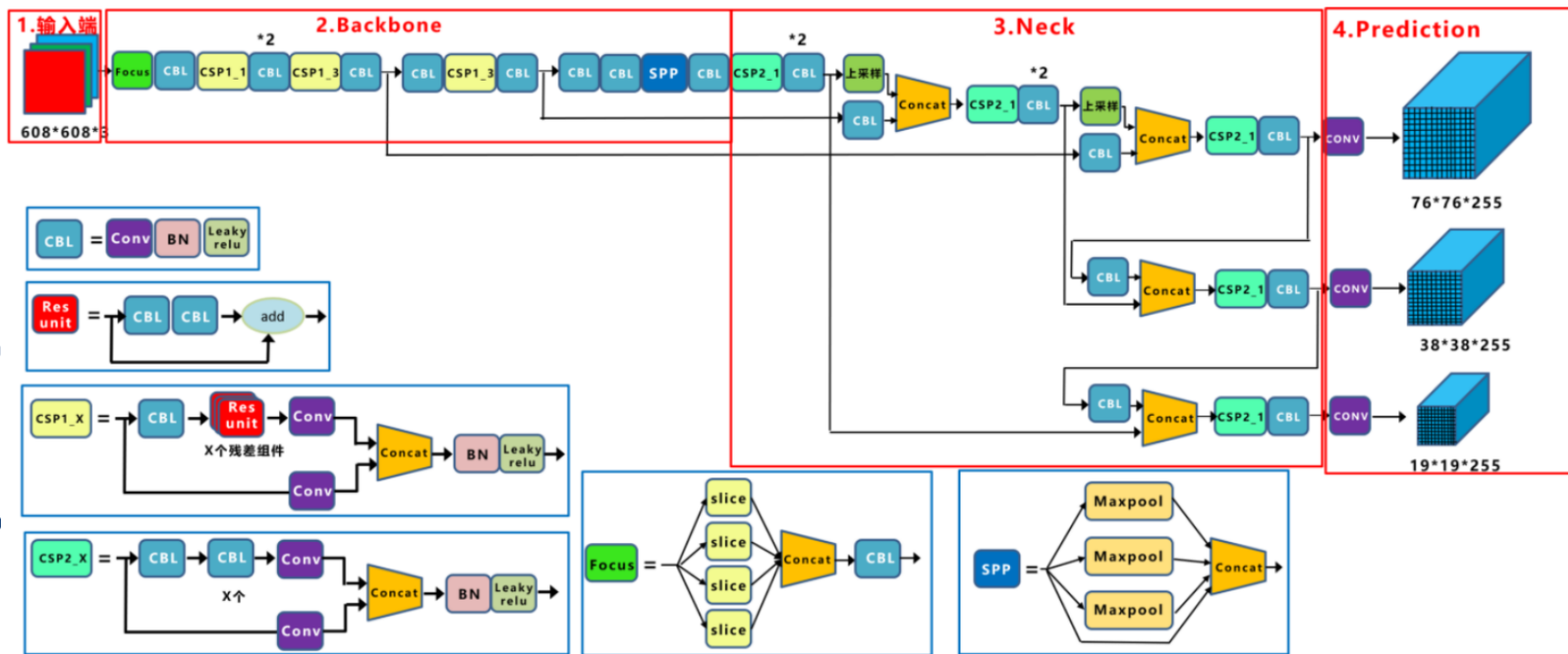


网络结构

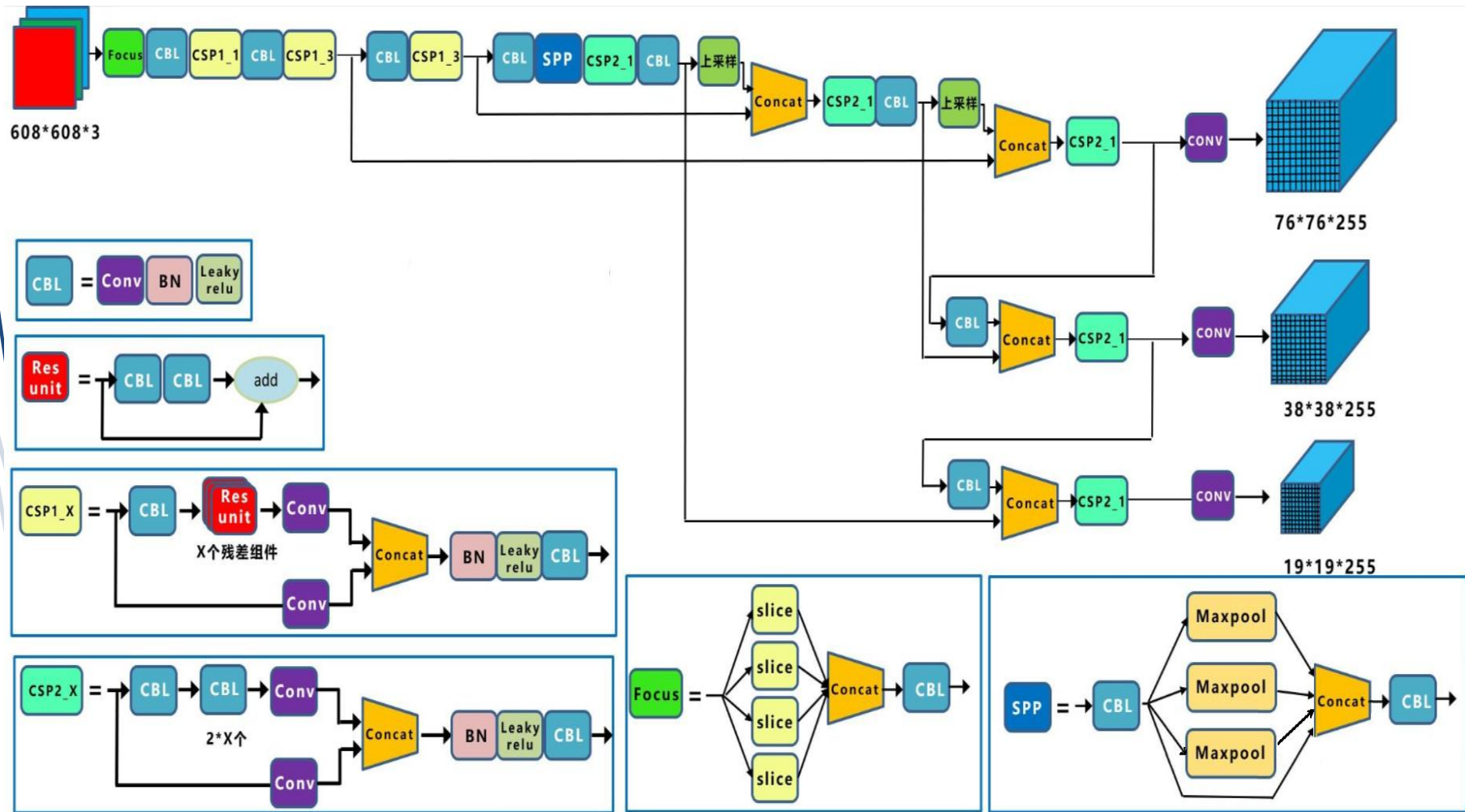
- ◆ **输入端**：训练时对输入端的改进，主要包括Mosaic数据增强，cmBN，SAT自对抗训练。
- ◆ **主干网络BackBone**：用各种新的方式结合起来，包括：CSPDarknet53，Mish激活函数，Dropblock
 - CSPDarknet53是在Yolov3主干网络Darknet53的基础上，借鉴CSPNet（2019）的经验，产生的Backbone结构，其中包含了5个CSP模块。
 - 每个CSP模块前面的卷积核大小都是3x3，步长为2，起到下采样的作用
- ◆ **Neck**：目标检测网络在BackBone和最后的输出层之间往往会插入一些层，比如Yolov4的SPP模块，FPN+PAN结构。
- ◆ **输出层Prediction**：锚框机制和Yolov3相同，主要改进的是训练时的损失函数CIoU_Loss，以及预测框筛选的nms变为DIoU_nms。

(6) YOLOv5 简介

- ◆ YOLOv5 模型是 Ultralytics 公司于 2020 年 6 月 9 日公开发布的。
- ◆ YOLOv5 基于 YOLOv3 模型基础改进而来
 - 有 YOLOv5s、YOLOv5m、YOLOv5l、YOLOv5x 四个模型
- ◆ YOLOv5 模型由输入端、Backbone、Neck、Prediction四个部分组成。



YOLOV5s Struture



网络特点

- (1) **输入端**: Mosaic数据增强、自适应锚框计算、自适应图片缩放
- (2) **Backbone**: Focus结构、CSP结构
- (3) **Neck**: FPN+PAN结构
- (4) **Prediction**: GIoU_Loss

(7) YOLOv6 简介

- ◆ YOLOv6 是由美团视觉智能部研发的一款目标检测框架，致力于工业应用。
- ◆ **YOLOv6于2024年发布**，其主要贡献包括：
 - 专注于检测的精度和推理效率，适用于工业界常用的尺寸模型。
 - 硬件友好性：考虑了硬件性能，优化了计算效率和内存使用，使其在GPU、CPU和ARM等不同平台高效运行。
 - 多平台支持：YOLOv6支持GPU（TensorRT）、CPU（OPENVINO）、ARM（MNN、TNN、NCNN）等不同平台的部署。
- ◆ **在精度和速度方面优越**，在COCO数据集上，YOLOv6的不同尺寸模型性能：
 - YOLOv6-nano：在COCO数据集上的精度为35.0% AP
 - YOLOv6-s：在COCO数据集上的精度为43.1% AP

(7) YOLOv6 简介

- ◆ YOLOv6的改进包括采用轻量级的卷积模块和引入注意力机制，更好地捕捉图像中的关键信息。
- ◆ 针对小目标检测问题进行了优化，改进了锚框的生成方式，适应不同尺寸的目标。
- ◆ YOLOv6在实际应用中，如自动驾驶和安防监控领域，展现出强大的潜力。
- ◆ YOLOv6的设计考虑了硬件感知，采用了RepVGG结构，设计了EfficientRep Backbone和Rep-PAN neck，以及Efficient Decoupled Head，进一步降低延时并提升精度

YOLOv6创新点

- ◆ YOLOv6在特征提取网络中引入了分层特征融合的思想，使得模型能够更好地学习和利用不同层次的特征信息。
- ◆ 采用了多尺度预测模块，增强了模型对不同大小目标的检测能力。
- ◆ YOLOv6采用新的损失函数设计，提高了模型对小目标的检测精度。

YOLOv6 简介

◆ YOLOv6检测算法的思路类似YOLOv5:

- (backbone+neck) +YOLOX (head)

◆ YOLOv6改进之处:

- 基于YOLOX设计, 检测头: 类似YOLOX, 进行解耦操作, 并进行优化:
 - 骨干网络: 由CSPDarknet换为了EfficientRep
 - neck: 基于Rep和PAN构建RepPAN, 采用FPN+RepPAN
 - 创新: 引入RepVGG, 为每一个3x3的卷积添加一个1x1的卷积分支和恒等映射分支; 推理阶段融合为3x3的结构

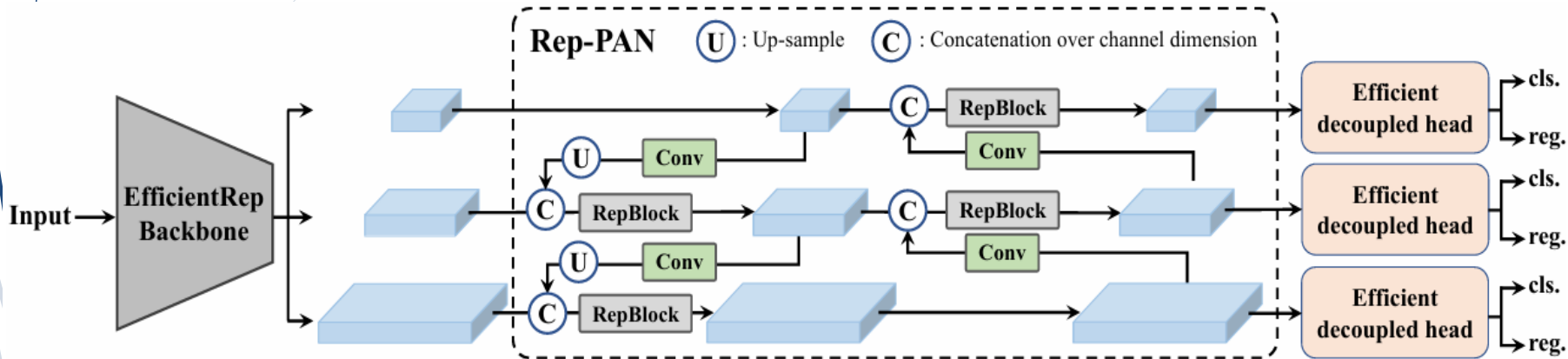


Figure 1 : The YOLOv6 framework (N and S are shown). Note for M/L, RepBlocks is replaced with CSPStackRep.

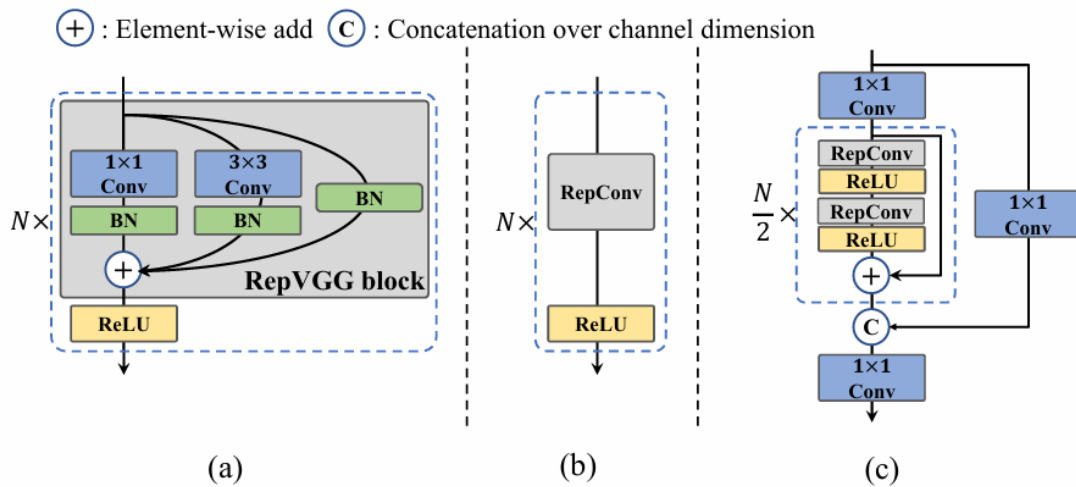
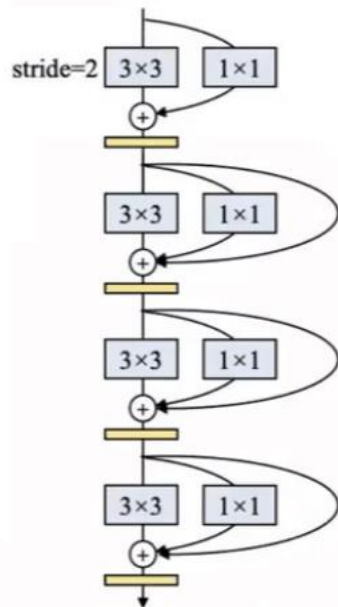
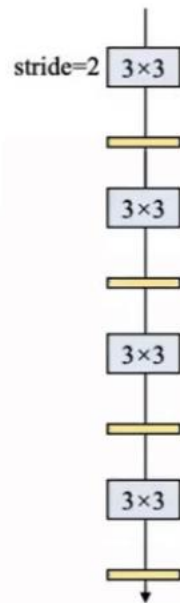


Figure 3: (a) RepBlock is composed of a stack of RepVGG blocks with ReLU activations at training. (b) During inference time, RepVGG block is converted to RepConv. (c) CSPStackRep Block comprises three 1×1 convolutional layers and a stack of sub-blocks of double RepConvs following the ReLU activations with a residual connection.

RepVGG style 结构



RepVGG training

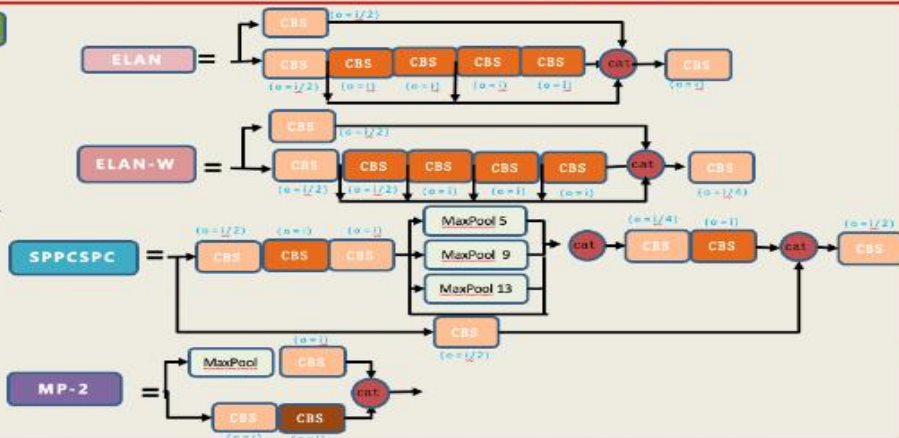
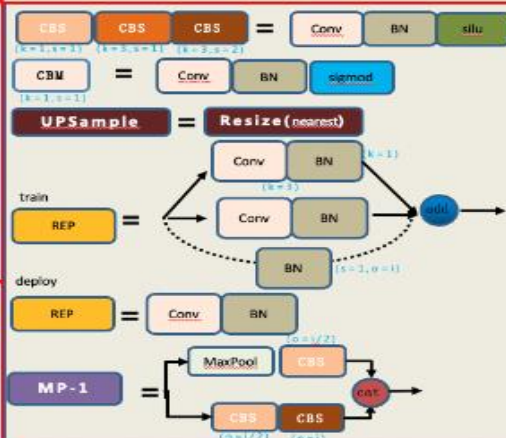
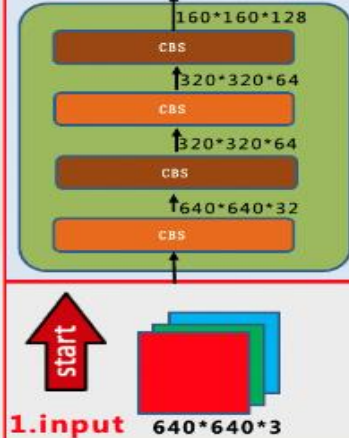


RepVGG inference

YOLOv7

- ◆ YOLOv7: 在YOLOv6的基础上, 进一步优化了分层特征融合策略和多尺度预测模块的设计。
- ◆ 采用了知识蒸馏技术, 将教师模型的先验知识传递给学生模型, 从而提高了模型的性能。
- ◆ 还优化了训练过程中的参数设置和数据增强策略, 提高了模型的泛化能力。
- ◆ ELAN高效网络架构: YOLOv7中提出的一个新的网络架构, 以高效为主。
- ◆ 带辅助头的训练: 通过增加训练成本, 提升精度, 同时不影响推理的时间, 辅助头只会出现在训练过程中。
- ◆ one-stage目标检测算法, YOLOv7在准确率和速度上较优越。

2.Backbone



YOLOv8

- ◆ YOLOv8由Joseph Redmon和Ali Farhadi在2015年首次提出。
- ◆ YOLOv8在2023年1月由ultralytics公司开源，支持全范围的视觉AI任务，包括分类、检测、分割、姿态估计和跟踪

YOLOv8

- ◆ YOLOv8: 作为YOLO系列的最新版本, YOLOv8在多个方面进行了改进和创新。
- ◆ 在特征提取网络中引入了新型结构设计, 如CSPNeck和SPP模块的改进等。
 - 采用了新的锚框生成策略和边框回归方法, 提高了模型的定位精度和鲁棒性。
 - 优化了损失函数设计, 并采用了新型训练策略如知识蒸馏和微调技术等
 - YOLOv8在准确率、速度和鲁棒性等方面均取得了显著提升。

- ◆ Ultralytics开发的YOLOv8是一款尖端、最先进（SOTA）的模型，借鉴之前YOLO版本的成功经验，并引入了新的特性和改进
- ◆ YOLOv8旨在实现快速、准确和易于使用，因此是各种目标检测、图像分割和图像分类任务的绝佳选择。
- ◆ YOLOv8支持最经典的目标检测任务，还添加了对语义分割、分类和追踪等任务的支持。

This is the YOLOv8 development timeline:

- **January 10, 2023:** YOLOv8 was officially released, featuring a new anchor-free architecture aimed at simplifying model training and improving detection accuracy across various tasks.
- **February 15, 2023:** Introduction of the YOLOv8 Python package and command-line interface (CLI), streamlining the process of model training, validation, and deployment.
- **March 5, 2023:** Implementation of advanced augmentation techniques such as mosaic and mixup augmentation, enhancing the model's ability to generalize across diverse datasets.
- **April 20, 2023:** Integration of the CSPNet backbone for improved feature extraction and a hybrid FPN+PAN neck, optimizing the model's performance in multi-scale object detection.
- **June 1, 2023:** Support for ONNX and TensorRT formats was added, facilitating deployment on a wider range of hardware platforms, including edge devices.

◆ YOLOv8算法的核心点

- 给出了一个全新的视觉模型，保持精度的同时，实现了较高的检测速度，并且同时支持支持图像分类、物体检测和实例分割等多种视觉任务。
- 并且提供了多个规模的模型（nano、small、medium、large和x-large），满足用户不同场景的需要。
- 新的骨干网络：YOLOv8引入了一个新的骨干网络，对不同尺度模型调整了不同的通道数，大幅提升了模型性能。
- 解耦头的设计：YOLOv8的Head部分从原先的耦合头变成了解耦头结构，将分类和检测头分离
- 新的损失函数：YOLOv8在Loss计算方面采用了TaskAlignedAssigner正样本分配策略，并引入了Distribution Focal Loss，确保了检测结果的准确性和鲁棒性。

YOLOv9

- ◆ YOLOv9由中国台湾Academia Sinica和台北科技大学等机构联合开发。
- ◆ YOLOv9的主要创新点包括提出了Programmable Gradient Information (PGI)结构和新的主干网络Generalized ELAN (GELAN)。
- ◆ PGI旨在解决深度网络传输中信息丢失的问题，通过可编程梯度信息来更新网络权重
- ◆ GELAN则是一种基于梯度路径规划的轻量级网络架构，能够支持任何计算块。

改进策略

- ◆ yolov9是在v7的基础上进行进一步改进
- ◆ 策略：搭配辅助检测头、隐性知识、锚框、固定标签分配与动态标签分配混合使用

YOLOv10

- ◆ YOLOv10的发展历程可以追溯到2016年，由Joseph Redmon和Ali Farhadi首次提出。
- ◆ YOLOv10是由清华大学的研究人员使用UltralyticsPython 软件包创建的：通过引入端到端头 (End-to-End head)，消除了非最大抑制 (NMS) 要求，实现了实时目标检测的进步，通过一致的双重分配策略，实现了高效的端到端检测，减少了推理延迟，提高了检测效率。
- ◆ 引入了整体效率精度驱动的设计策略，改善了性能效率的权衡。
- ◆ YOLOv10在CSPNet的基础上进行了增强，引入了PAN（路径聚合网络）层，实现了多尺度特征融合。
- ◆ YOLOv10在实时目标检测中表现出色，适用于各种应用场景。

YOLOv11

- ◆ YOLOv11是YOLO (You Only Look Once) 系列的最新版本，由Ultralytics团队开发。
- ◆ YOLO Vision 2024 (YV24) 大会上，Ultralytics 正式发布了YOLO11
- ◆ 性能飞跃：在COCO数据集上，YOLO11m比YOLOv8m使用22%更少的参数，却实现了更高的平均精度mAP。
- ◆ 速度提升：推理速度比YOLOv10快约2%，为实时应用提供了更好的支持。
- ◆ 多功能性：支持目标检测、实例分割、图像分类、姿态估计、定向目标检测（OBB）和目标跟踪等多种任务。
- ◆ 增强的特征提取：改进的架构设计使得YOLO11能够更准确地捕捉图像中的复杂细节。
- ◆ 优化的训练方法：增强了模型的适应性，使其更容易应用于不同规模的项目

小结：YOLO系列算法

- ◆ 实现直接回归功能，不需要复杂的设计过程，直接用一整张图进行训练模型，能够更好的区分目标和背景区域。
 - 同时预测多个Box位置和类别
 - 端到端的目标检测和识别
 - 速度更快

yolo系列网络优点

- ◆ 快速，框架搭建简单
- ◆ 可以快速识别图像中的目标，但它在定位某些物体尤其是小的物体上精度不高。
- ◆ 与RCNN系列物识别物体相比，位置精度准确性差

补充资料

◆ DeepSDF

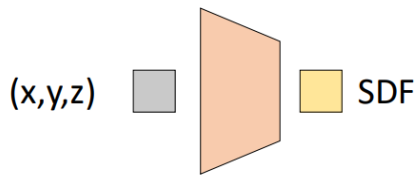
DeepSDF

◆ 2019CVPR顶级会议文章:

Jeong Joon Park, Peter R. Florence, Julian Straub, Richard A. Newcombe, Steven Lovegrove:
DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation. CVPR 2019:
165-174

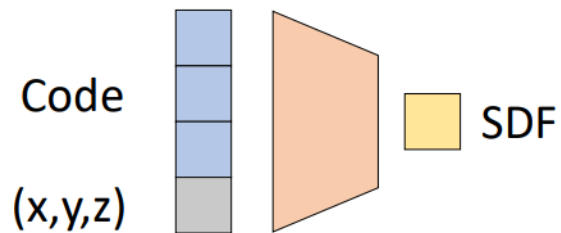
主要思想:

- ◆ 用SDF (Signed Distance Function) 函数来隐式表示三维物体
- ◆ 输入是三维空间点, 输出三维空间点的符号距离函数



Single Shape DeepSDF

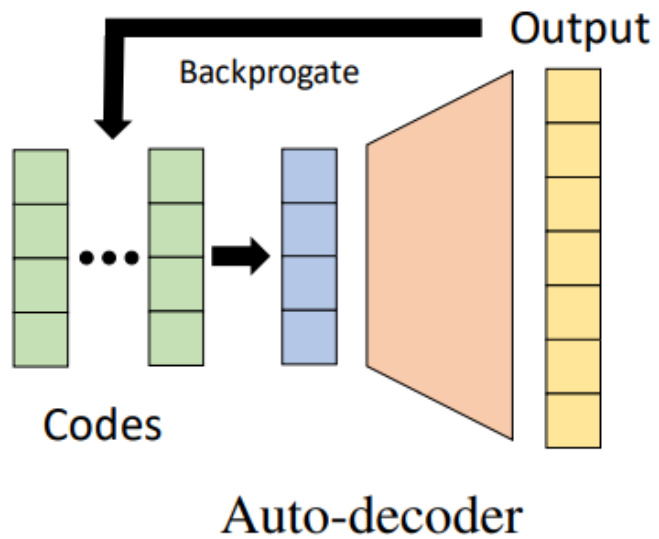
系统结构



Coded Shape DeepSDF

Encoder-decoder 结构

- ◆ 在训练阶段可以用编码-解码网络
- ◆ 实践应用时，仅仅使用解码器：固定解码器的参数，反向传播，优化中间的 latent code
- ◆ 采用 auto-decoder 的架构
- ◆ 训练阶段：A randomly initialized latent vector is assigned to each data point in the beginning of training, and the latent vectors are optimized along with the decoder weights through standard backpropagation.
- ◆ 预测阶段：During inference, decoder weights are fixed, and an optimal latent vector is estimated.



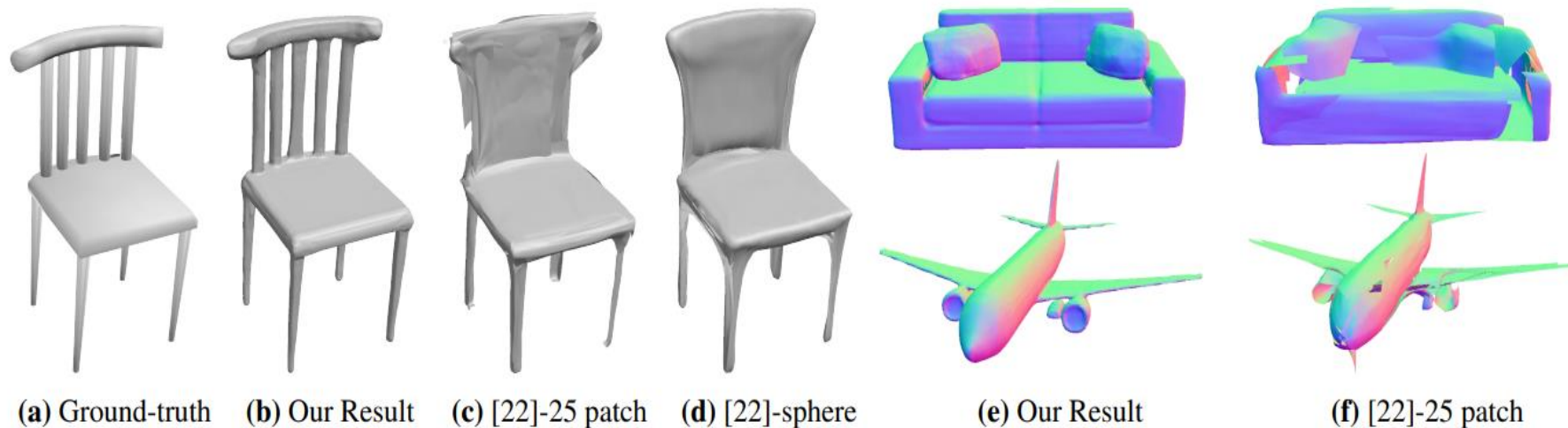


Figure 6: Reconstruction comparison between DeepSDF and AtlasNet [22] (with 25-plane and sphere parameterization) for test shapes. Note that AtlasNet fails to capture the fine details of the chair, and that (f) shows holes on the surface of sofa and the plane.

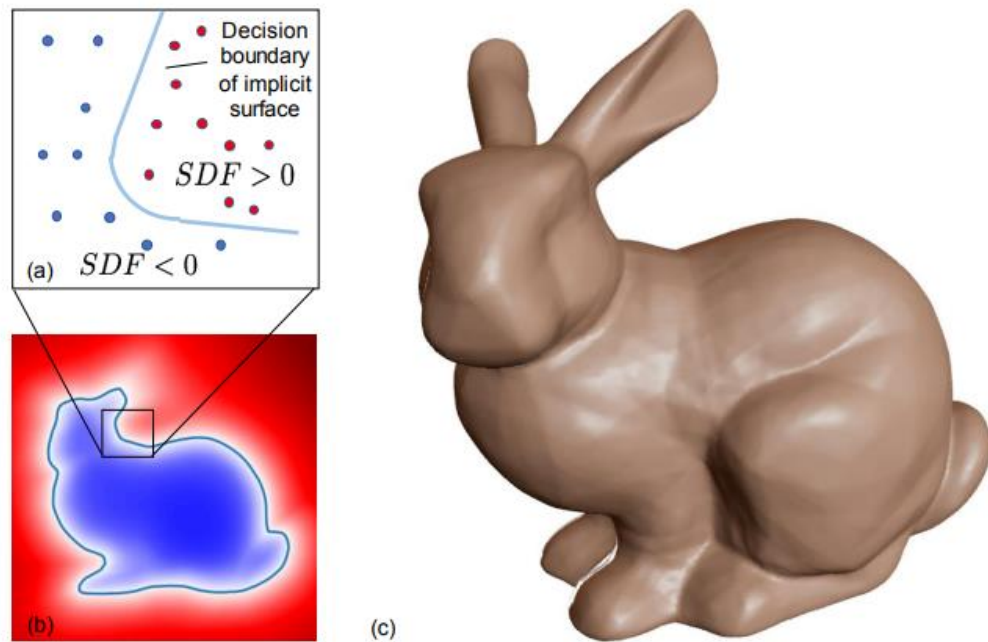


Figure 2: Our DeepSDF representation applied to the Stanford Bunny: (a) depiction of the underlying implicit surface $SDF = 0$ trained on sampled points inside $SDF < 0$ and outside $SDF > 0$ the surface, (b) 2D cross-section of the signed distance field, (c) rendered 3D surface recovered from $SDF = 0$. Note that (b) and (c) are recovered via DeepSDF.